Manual Funcional del Sistema — Modular Learning

Versión del documento: 2.1 (actualizado tras P19–P29)
Alcance: Especificación funcional completa basada en el comportamiento real del código fuente (`backend/`, `frontend/`, `scripts/`, `deploy/`), contrastada con la documentación del repositorio (`README.md`, `docs/ARCHITECTURE.md`, `docs/DESPLIEGUE.md`) y los criterios funcionales verificados hasta `P29` y `D3`.

---

1. Qué es y para quién

1.1. Propósito del sistema

Modular Learning es un asistente académico e interactivo de estudio universitario diseñado bajo **Arquitectura Hexagonal (Puertos y Adaptadores)** e impulsado por **Google Gemini** (`google-genai`, modelo por defecto `gemini-3.5-flash-lite` definido en `backend/src/modular_learning/infrastructure/llm/gemini_adapter.py:29`). Su objetivo es acompañar al estudiante durante todo el semestre en seis actividades centrales:

1. **Planificación curricular:** organizar el catálogo de asignaturas y extraer cronogramas de clases desde sílabos oficiales (`.pdf` o `.docx`).
2. **Ingesta y alineación de material:** convertir lecturas, diapositivas, guías de actividad y proyectos prácticos en apuntes estructurados en Markdown dentro de una bóveda de **Obsidian**, vinculándolos automáticamente a la sesión correspondiente del cronograma.
3. **Tutoría socrática interactiva:** guiar el estudio de un tema paso a paso mediante explicaciones en *streaming* (Server-Sent Events), resolución de dudas con contexto curricular y práctico, y generación de un resumen final en Obsidian.
4. **Estudio fuera de línea (Guías Offline):** generar cuadernos de estudio en formato **PDF A5** listos para imprimir o leer sin conexión, y registrar al regreso qué clases quedaron entendidas o con dudas.
5. **Evaluación activa y repaso espaciado:** generar cuestionarios (*quizzes*) de opción múltiple y preguntas abiertas evaluadas con IA, programando tarjetas de repaso bajo una variante del algoritmo **SM-2**.
6. **Contexto práctico mediante plugins:** analizar laboratorios y código fuente de proyectos reales (**Cisco Packet Tracer**, **Node-RED**, **Desarrollo Web**) para inyectar su arquitectura y estado en las explicaciones del tutor.

1.2. Destinatarios de este manual

- **Estudiante (usuario final):** utiliza la interfaz web o la consola para gestionar su semestre, ingestar apuntes, estudiar con el tutor o en papel y repasar antes de los parciales.
- **Revisor técnico** / **mantenedor:** verifica qué hace exactamente cada vista, endpoint y comando del sistema, dónde persiste cada dato y qué discrepancias existen entre el comportamiento esperado y la implementación actual.

1.3. Interfaces del sistema

El núcleo de aplicación (`backend/src/modular_learning/core/`) se expone a través de dos adaptadores primarios que comparten el mismo contenedor de dependencias (`Container` y `build_container` en `backend/src/modular_learning/bootstrap.py:57-246`):

1. **Aplicación Web (`ml-web` + SPA React):**
   - **Backend HTTP:** servidor FastAPI (`backend/src/modular_learning/apps/api/main.py:38-148`) iniciado con el entrypoint `ml-web` (`backend/pyproject.toml:34`, puerto `8000` por defecto). Expone 55 rutas bajo `/api/*` y sirve la SPA compilada en `backend/src/modular_learning/apps/api/static/` (`apps/api/main.py:35, 76-132`).
   - **Frontend SPA:** aplicación React 19 + TypeScript + Vite (`frontend/src/App.tsx:13-65` y `frontend/src/components/layout/Sidebar.tsx:11-165`) organizada en un **menú lateral de 5 entradas**:
     1. **Hoy** (`/` → `HoyView.tsx:60-386`): clase del día o próxima, guías pendientes de confirmar, repasos vencidos, material nuevo por convertir (en modo local/híbrido) y tarjetas de asignaturas con barra de avance y desplegable «Ver progreso completo».
     2. **Estudiar** (`/estudiar` → `EstudiarView.tsx:16-71`): contenedor con las pestañas **Tutoría** (`?tab=tutoria` → `TutoriaView.tsx:65-697`), **Repaso** (`?tab=repaso` → `QuizView.tsx:32-559`) y **Guía** (`?tab=guia` → `GuiaView.tsx:207-612`).
     3. **Calendario** (`/calendario` → `CalendarioView.tsx:57-241`): cuadrícula mensual con navegación entre meses y panel de detalle del día seleccionado.
     4. **Asignaturas** (`/asignaturas` → `AsignaturasView.tsx:22-294`): selector único compartido de asignatura (con creación y renombrado) y tres pestañas: **Clases** (`SilabosView.tsx:38-1226` embebido), **Material** (`MaterialView.tsx:120-1279` en `modo="materiales"`) y **Proyectos** (`MaterialView.tsx` en `modo="proyectos"`), con secciones colapsables «Zona peligrosa» al pie de **Clases** y **Material**.
     5. **Ajustes** (`/ajustes` → `AjustesView.tsx:14-63`): contenedor con las pestañas **Carpetas y clave** (`?tab=config` → `ConfigView.tsx:23-221`) y **Plugins** (`?tab=plugins` → `PluginsView.tsx:45-345`).
   - **Vistas de guías y redirecciones heredadas:** incluye `/guia/:id/revision` (`RevisionGuiaView.tsx:119-304`) y `/guia/:id/imprimir` (`GuiaImprimirView.tsx:21-157`, sin barra lateral). Las rutas anteriores (`/tutoria`, `/repaso`, `/quiz`, `/guia`, `/progreso`, `/plugins`, `/config`, `/silabos`, `/material`) redirigen automáticamente a su ubicación en el menú de 5 entradas conservando los parámetros de búsqueda (`RedirigirConQuery` en `frontend/src/App.tsx:13-19, 50-59`).

2. **Interfaz de Línea de Comandos (`ml`):**
   - **CLI con Typer y Rich:** definida en `backend/src/modular_learning/apps/cli/main.py:10-77` (`backend/pyproject.toml:33`). Ofrece los subcomandos `ml init`, `ml es`, `ml im`, `ml quiz`, `ml stats` y `ml up`, además del módulo ejecutable `apps/cli/commands/snapshot.py` y los scripts operativos `scripts/migrar_datos.py` y `deploy/subir_datos.py`.

1.4. Modos de operación: Local, Nube e Híbrido

El comportamiento de persistencia, seguridad y acceso al sistema de archivos depende de las variables de entorno leídas en `infrastructure/persistence/almacen.py:172-182`, `apps/api/routers/config.py:21` y `apps/api/acceso.py:23-73` (además de la opción de contenedor local con PostgreSQL en `docker-compose.yml`, validada en `P18` — commit `e171812`, y la caché en memoria de `AlmacenPostgres` añadida en `P19` — commit `30980dc`):

Característica	Modo Local	Modo en la Nube (Render + Neon)	Modo Híbrido (PC local + Neon)
Variables clave	Sin `DATABASE_URL`; sin `ML_EN_NUBE`	`DATABASE_URL=postgresql://...`, `ML_EN_NUBE=1`, `ML_REQUIERE_ACCESO=1`	`DATABASE_URL=postgresql://...`, sin `ML_EN_NUBE` (o `ML_EN_NUBE=0`)
Almacén de documentos (`AlmacenDocumentos`)	`AlmacenArchivos` (`infrastructure/persistence/almacen.py:47-95`) sobre `data/` (o `ML_DATA_DIR`)	`AlmacenPostgres` (`infrastructure/persistence/almacen.py:98-165`) en la tabla `documentos` de Neon con caché TTL (`ML_CACHE_TTL_SEGUNDOS`, por defecto `10` s)	`AlmacenPostgres` conectado a la misma tabla `documentos` de Neon
Acceso a carpetas locales de sílabos, material y proyectos	Sí (`LectorDocumentosLocal`, `InspectorProyectosLocal`, plugins)	No escanea carpetas locales: `HoyView.tsx:113-131` y `MaterialView.tsx:222, 591` omiten el escaneo cuando `en_nube` es `true`, y `/api/material/escanear` (`routers/material.py:57-58`) y `/api/proyectos/escanear` (`routers/proyectos.py:37-38`) devuelven listas vacías (`P25`, commit `7da3dab`)	Sí: lee los archivos locales del PC mientras lee/escribe el estado JSON en Neon
Bóveda de Obsidian	Escribe directamente en la carpeta local configurada en `obsidian_vault`	Escribe en `/app/data/boveda` dentro del contenedor (efímero entre reinicios de Render; `deploy/subir_datos.py:27-30`)	Escribe en la bóveda local de Obsidian si `obsidian_vault` apunta a una ruta local válida
Autenticación HTTP	Desactivada por defecto (`ML_REQUIERE_ACCESO` no activo)	HTTP Basic obligatorio en todas las rutas salvo `/api/salud` (`apps/api/acceso.py:34-73`) usando `ML_USUARIO` y `ML_CONTRASENA`	Opcional (habitualmente desactivada en `localhost`)
Interfaz de Ajustes (`ConfigView`)	Permite editar rutas locales y clave de Gemini (`frontend/src/views/ConfigView.tsx:110-217`)	Muestra el aviso «Entorno en la nube activo» y deshabilita la edición de carpetas y clave (`ConfigView.tsx:129-144, 169, 188, 211`)	Permite editar ajustes (cuidado: pisa `config/ajustes.json` en Postgres si se guarda)

2. Conceptos del dominio

2.1. Asignatura (`Materia`: `slug` y `nombre`)

- **Definición:** entidad `Materia(slug: str, nombre: str)` en `backend/src/modular_learning/core/domain/materias.py:23-41`.
- **Cálculo del `slug`:** la función `slug_materia(texto)` (`core/domain/materias.py:11-20`) descompone caracteres Unicode (`NFKD`), elimina marcas diacríticas (tildes/diéresis), convierte a minúsculas y reemplaza cualquier secuencia no alfanumérica por un guion `-`. Es idempotente: `slug_materia("Ingeniería de Software II") == slug_materia("ingenieria-de-software-ii") == "ingenieria-de-software-ii"`.
- **En el frontend:** `clave(texto)` en `frontend/src/lib/asignaturas.ts:7-14` aplica la misma normalización para cruzar nombres con el catálogo (`buscarMateriaPorClave` y `resolverMateriaSubida` en `asignaturas.ts:106-143`).
- **Propósito:** el `slug` es la clave primaria e invariante para nombrar archivos de cronogramas (`schedules/<slug>_cronograma.json`) y referenciar la asignatura en URLs; el `nombre` es la etiqueta legible con mayúsculas y tildes que se muestra en la interfaz y nombra las carpetas dentro de Obsidian.

2.2. Cronograma

- **Definición:** documento JSON por asignatura almacenado bajo la clave `schedules/<slug>_cronograma.json` (`infrastructure/persistence/json_repositorios.py:103-112`).
- **Estructura:** contiene `materia` (nombre visible), `clases` (lista ordenada de diccionarios de clase), `parciales` (lista opcional de cortes de parcial) y `material_transversal` (lista opcional de nombres de materiales que aplican a toda la asignatura).

2.3. Clase (`id` y `estado`)

- **Definición:** cada entrada de la lista `clases` de un cronograma (`core/domain/cronograma.py:13-77`).
- **Identificador (`id`):** cadena hexadecimal de 8 caracteres generada con `uuid.uuid4().hex[:8]` por `nuevo_id_clase()` (`core/domain/cronograma.py:13-14`). `asignar_ids()` (`cronograma.py:17-34`) garantiza que toda clase tenga un `id` único dentro del cronograma al cargar o guardar.
- **Atributos:**
  - `id`: identificador único de la sesión.
  - `tema`: título del tema de la clase.
  - `fecha`: cadena con la fecha o semana (ej. `"14/04/2026"` o `"Semana 1"`).
  - `objetivo`: competencia u objetivo pedagógico de la sesión.
  - `contenido`: lista de subtemas (`list[str]`) o cadena descriptiva.
  - `estado`: `"pendiente"` (por defecto) o `"completado"`.
  - `fecha_completado`: fecha ISO (`YYYY-MM-DD`) añadida cuando la clase pasa a `"completado"` (`json_repositorios.py:254-255`).
  - `materiales_asociados`: lista de nombres de archivos ingestados vinculados a esta clase por el alineador curricular (`core/domain/alineador_curricular.py:92-96`).
  - `subtemas_adicionales`: lista de subtemas extraídos de los materiales ingestados e inyectados en la clase (`alineador_curricular.py:98-103`).

2.4. Parcial (Corte de parcial)

- **Definición:** marcador dentro del campo `parciales` del cronograma (`[{"nombre": "Parcial 1", "hasta": "<id_clase>"}]`), gestionado por `JsonCronogramaRepo.obtener_parciales` y `guardar_parciales` (`json_repositorios.py:307-329`).
- **Asociación por clase (`P27`, commit `2cead9e`):** al definir un corte en **Asignaturas › Clases** (`frontend/src/views/SilabosView.tsx:745-777`), el estudiante elige en un desplegable `<select>` la última clase del cronograma que entra en ese parcial y se guarda su `id` en `hasta` (o su índice `1..N` si no tuviera `id`). Las utilidades `resolverIndiceHasta` y `describirCorte` (`frontend/src/lib/parciales.ts:19-67`) resuelven el índice de la clase límite tanto por `id` como por número de orden o fecha en texto libre (retrocompatibilidad), permitiendo seleccionar de un clic todas las clases pendientes del corte al armar una guía offline en **Estudiar › Guía** (`frontend/src/views/GuiaView.tsx:345-363, 456-464`).

2.5. Material de estudio y alineación curricular

- **Definición:** archivo académico procesado por `IngestaService` (`core/application/ingesta.py:29-407`).
- **Formatos admitidos:** `.pdf`, `.docx`, `.txt`, `.md` y `.pptx` (`EXTENSIONES_MATERIAL` en `ingesta.py:29`, actualizado en `P16` — commit `d40428d`, con `EXTENSIONES_NO_SOPORTADAS_WEB = ()` en `ingesta.py:30`).
- **Procesamiento de presentaciones `.pptx` (`IngestaService._generar_contenido`, `ingesta.py:94-124`, completado en `P16 d40428d` y `P24 faf23ac`):**
  - **Vía principal (con LibreOffice):** si el puerto `ConversorPdfPort` (`ConversorLibreOffice` en `infrastructure/documents/conversor_libreoffice.py:40-112`) encuentra `soffice` (`ML_SOFFICE`, `PATH` o rutas estándar), crea un directorio de perfil temporal aislado (`-env:UserInstallation=file:///...`, líneas `62-75`), convierte el `.pptx` a un PDF temporal en modo `--headless`, lo envía a Gemini como documento multimodal (`generar_texto_con_archivo`) para analizar texto, imágenes y diagramas, y borra el PDF y el perfil temporal al terminar (`ingesta.py:99-107`, `conversor_libreoffice.py:97, 100-112`).
  - **Plan B (sin LibreOffice):** extrae el texto de diapositivas, tablas y notas del orador mediante `python-pptx` en `LectorDocumentosLocal.extraer_texto` (`infrastructure/documents/lector_documentos.py:56-84`). Si el texto útil extraído (descontando separadores `--- Diapositiva N ---`) tiene menos de `MIN_CARACTERES_PPTX_TEXTO = 200` caracteres (`ingesta.py:39, 111-120`), lanza `ValueError` indicando que la presentación contiene principalmente imágenes y requiere instalar LibreOffice; en caso contrario, envía el texto a Gemini y devuelve el campo `aviso` en el resultado (`ingesta.py:122-124, 226-227`), que la interfaz web muestra como advertencia (`MaterialView.tsx:396-403`).
- **Clasificación automática por ruta:** `IngestaService.clasificar_tipo_material` (`ingesta.py:68-83`) inspecciona la ruta relativa dentro de la carpeta de la asignatura:
  - En la raíz de la asignatura o en subcarpeta `ACTIVIDADES` / `ACTIVIDAD` → tipo `actividad`.
  - Contiene `"MATERIAL DE ESTUDIO"` → tipo `diapositiva`.
  - Contiene `"MATERIAL ADICIONAL"` → tipo `lectura`.
  - Contiene `"PROYECTO"` / `"PROYECTOS"` → tipo `proyecto` (delegado a `ProyectosService`).
  - Cualquier otro caso (o valor por defecto en subida individual) → tipo `material_complementario`.
- **Alineación curricular:** al ingestar un archivo, `IngestaService._alinear_con_cronograma` (`ingesta.py:134-150`) delega en `asignar_material_a_clase` (`core/domain/alineador_curricular.py:51-109`):
  - Si el documento es transversal (`asignado is False`), lo registra con `JsonCronogramaRepo.agregar_material_transversal` (`json_repositorios.py:271-281`).
  - Si coincide con una clase del cronograma (`asignado is True`), vincula el archivo en `materiales_asociados`, inyecta sus `subtemas_adicionales` en esa clase y guarda el cronograma actualizado.

2.6. Nota en Obsidian

- **Definición:** archivo Markdown (`.md`) escrito en el sistema de archivos local a través del puerto `BovedaNotasPort` (`infrastructure/obsidian/boveda_obsidian.py:12-136`).
- **Tipos de notas generadas:**
  1. **Apunte de material ingestado:** `<obsidian_vault>/<materia>/<SubcarpetaTipo>/<nombre_base>_Notas.md` (`ingesta.py:173-179`).
  2. **Ficha técnica de proyecto:** `<obsidian_vault>/<materia>/Proyectos/<nombre_limpio>.md` (`proyectos.py:149-153`).
  3. **Resumen de sesión de tutoría:** `<obsidian_vault>/<materia>/<tema_limpio>.md` con *frontmatter* YAML (`tags`, `fecha`, `estado: completado`) y enlace bidireccional en el índice `<obsidian_vault>/<materia>.md` (`boveda_obsidian.py:44-120`).

2.7. Proyecto

- **Definición:** entidad `Proyecto` (`core/domain/proyectos.py:39-64`) que representa un entregable práctico o documental de varias semanas (un repositorio de código, una topología IoT o un informe de auditoría) ubicado dentro de `<material_dir>/<Asignatura>/Proyectos/...`.
- **Detección e inspección:** `InspectorProyectosLocal` (`infrastructure/proyectos/inspector_local.py:55-186`) detecta carpetas de proyecto saltando directorios contenedores (`proyectos`, `general`, `primer_parcial`, `segundo_parcial`, etc.) y clasifica cada proyecto en uno de cuatro tipos (`detectar_tipo`, líneas `129-147`):
  - `software`: contiene archivos `.py`, `.ts`, `.tsx`, `.js`, `.java`, `.cs`, `.go` o `.sql`.
  - `iot`: contiene flujos `.json` o archivos `.ino`.
  - `documental_auditoria`: contiene más de un `.docx`/`.pdf` o su ruta incluye `"auditoria"` / `"informe"`.
  - `documento_rector`: caso por defecto cuando solo contiene un documento base.
- **Parcial asociado:** deducido de los nombres de carpeta en la ruta (`_parcial_de_carpeta`, `inspector_local.py:46-52`; `etiqueta_parcial`, `core/domain/proyectos.py:23-27`): `"primer_parcial"`, `"segundo_parcial"` o `"general"`.
- **Huella de cambio (`HuellaProyecto`):** almacena `total_archivos` y `ultimo_mtime` (`core/domain/proyectos.py:30-36`). Se considera modificado cuando cambia el número de archivos o cuando `ultimo_mtime` supera el registrado por más de `TOLERANCIA_MTIME_SEGUNDOS = 2.0` segundos (`core/application/proyectos.py:22, 184-187`).

2.8. Guía offline

- **Definición:** entidad `GuiaOffline` (`core/domain/guia_offline.py:83-131`) compuesta por una lista ordenada de `ClaseGuia` (`guia_offline.py:65-80`). Permite estudiar un conjunto de clases fuera de la pantalla (impresa en cuadernillo A5 o en lector PDF).
- **Nivel de extensión (`ExtensionGuia`):** determina la profundidad del prompt enviado a Gemini (`core/application/guias.py:20-26`) y se elige automáticamente (`elegir_extension`, `guia_offline.py:28-53`):
  - Por presupuesto de tiempo (`tiempo_minutos`): `<= 45` min → `"sintesis"`, `<= 90` min → `"estandar"`, `> 90` min → `"completa"` (acotada por el número de clases).
  - Por cantidad de clases seleccionadas (si no se indican minutos): `1-2` clases → `"completa"` (desarrollo completo + 3 preguntas), `3-5` clases → `"estandar"` (350-500 palabras + 2 preguntas), `6+` clases → `"sintesis"` (200-300 palabras + 1 pregunta).
- **Separación de respuestas:** cada clase generada incluye el marcador `<!-- RESPUESTAS -->` (`MARCADOR_RESPUESTAS`, `guia_offline.py:13`). Al exportar a PDF o imprimir, el cuerpo de estudio (`cuerpo`) se coloca en la sección de lectura y el bloque de soluciones (`respuestas`) se agrupa al final del documento en un «Solucionario» (`infrastructure/pdf/exportador_pdf.py:67-86`).
- **Ciclo de revisión y reversión:** cada `ClaseGuia` guarda `estado_previo` (el estado que tenía la clase en el cronograma antes de confirmar la guía) y `resultado` (`"pendiente"`, `"entendida"`, `"con_dudas"`). Si se marca `"entendida"`, la clase pasa a `"completado"` en el cronograma y se registra en el repaso espaciado; si se deshace la revisión, cada clase vuelve a su `estado_previo` (`core/application/guias.py:157-250`).

2.9. Repaso espaciado y tarjeta (`Tarjeta`)

- **Definición:** entidad `Tarjeta` (`core/domain/repaso_espaciado.py:36-71`) generada a partir de cada pregunta respondida en un quiz o al cerrar una tutoría / guía offline.
- **Identificador determinista:** `id_tarjeta(materia, enunciado)` calcula los primeros 12 caracteres del hash SHA-1 de `slug_materia(materia)::enunciado.lower()` (`repaso_espaciado.py:21-26`), evitando duplicar tarjetas para la misma pregunta.
- **Algoritmo de programación (`programar`):** implementado en `core/domain/repaso_espaciado.py:74-108` (variante SM-2):
  - **Fallo (`correcto = False`):** `racha = 0`, `intervalo_dias = 1` (vence mañana), `facilidad = max(1.3, facilidad - 0.2)`.
  - **Acierto (`correcto = True`):** incrementa `racha`; si `racha == 1` → `intervalo_dias = 1`; si `racha == 2` → `intervalo_dias = 3`; si `racha >= 3` → `intervalo_dias = max(4, round(intervalo_anterior * facilidad))` y `facilidad = min(2.8, facilidad + 0.05)`.
  - **Tarjeta dominada:** cuando `intervalo_dias >= INTERVALO_DOMINADA` (`21` días, línea 15), `Tarjeta.dominada` es `True` (`repaso_espaciado.py:58-59`).
  - **Límite diario:** `LIMITE_DIARIO = 10` preguntas por sesión (`repaso_espaciado.py:16`), priorizando las tarjetas con último fallo y fecha `proximo_repaso` más antigua (`core/application/repaso.py:122-126`).

2.10. Plugin práctico

- **Definición:** adaptador que hereda de `PluginPracticoBase` (`core/ports/plugin_practico.py:7-44`) para extraer contexto técnico

de herramientas de laboratorio y guardarlo como memoria Markdown en `plugins/<id>_context.md`.

- **Plugins incluidos en el sistema (`bootstrap.py:143-147`):**
  1. `cisco` (`CiscoPlugin` en `infrastructure/plugins/cisco.py:213-388`): descomprime y analiza archivos `.pkt` / `.pka` de Cisco Packet Tracer (con `zlib` y heurísticas binarias/XML) y documentos `.pdf`/`.docx` adjuntos para extraer dispositivos, VLANs, direccionamiento IP y rúbricas. Materia por defecto: `"Conmutación y Enrutamiento"`.
  2. `nodered` (`NodeRedPlugin` en `infrastructure/plugins/nodered.py:31-245`): analiza flujos JSON de Node-RED identificando nodos MQTT, HTTP, funciones JavaScript y dashboards. Materia por defecto: `"Internet de las Cosas"`.
  3. `web` (`WebPlugin` en `infrastructure/plugins/web.py:304-429`): inspecciona proyectos full-stack (`.py`, `.ts`, `.tsx`, `.js`, `.html`, `.css`, `package.json`, `requirements.txt`), resumiendo su árbol de directorios, stack tecnológico y esqueleto de clases/funciones. Materia por defecto: `"Ingeniería de Software II"`.
- **Enrutamiento al tutor (`PluginsService.obtener_plugin`):** implementado en `core/application/plugins.py:125-160`, busca coincidencia en tres niveles: 1) configuración explícita en `config/plugins_config.json`, 2) `materia_asociada_defecto` del plugin (comparando tanto nombre normalizado como `slug_materia`), y 3) palabras clave en el nombre de la asignatura o del tema.

---

3. Datos y almacenamiento

3.1. Abstracción `AlmacenDocumentos`

Toda la persistencia de configuración y estado del sistema pasa por el puerto `AlmacenDocumentos` (`infrastructure/persistence/almacen.py:27-44`), que expone cuatro operaciones sobre claves lógicas con formato de ruta relativa (`leer`, `escribir`, `eliminar`, `listar`):

- **En modo local (`AlmacenArchivos`, `almacen.py:47-95`):** cada clave `<clave>` corresponde al archivo físico `<ML_DATA_DIR o data>/<clave>` en disco, con escritura atómica mediante archivo temporal `.tmp` y `os.replace`.
- **En modo nube o híbrido (`AlmacenPostgres`, `almacen.py:98-165`):** cada clave `<clave>` corresponde a una fila en la tabla PostgreSQL (con caché en memoria de lecturas controlada por `ML_CACHE_TTL_SEGUNDOS`, implementada en `P19` — commit `30980dc`):

```
CREATE TABLE IF NOT EXISTS documentos (
    clave TEXT PRIMARY KEY,
    contenido TEXT NOT NULL,
    actualizado TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
```

3.2. Inventario de documentos persistidos en `AlmacenDocumentos`

Clave en `AlmacenDocumentos` (ruta en `data/` o `clave` en tabla `documentos`)	Qué guarda	Estructura principal	Quién lo lee y escribe
`config/ajustes.json`	Rutas locales de trabajo (`carpeta_material`, `ruta_obsidian`) y `gemini_api_key` opcional	`{"carpeta_material": str, "ruta_obsidian": str, "gemini_api_key": str}`	`JsonAjustesRepo` (`infrastructure/persistence/json_ajustes.py:15-66`), `deploy/subir_datos.py:27-30`
`config/materias.json`	Catálogo oficial de asignaturas del estudiante	`{"materias": [{"slug": str, "nombre": str}, ...]}`	`JsonMateriaRepo` (`infrastructure/persistence/json_repositorios.py:29-95`)
`schedules/<slug>_cronograma.json`	Cronograma de clases, cortes de parcial y material transversal de una asignatura	`{"materia": str, "clases": [...], "parciales": [...], "material_transversal": [...]}`	`JsonCronogramaRepo` (`infrastructure/persistence/json_repositorios.py:97-329`)
`config/registro_materiales.json`	Historial de archivos y proyectos ingestados para evitar re-procesarlos	`[{"archivo": str, "materia": str, "tipo": str, "nota": str, "fecha": str, ...}]`	`JsonRegistroMaterialesRepo` (`infrastructure/persistence/json_repositorios.py:332-399`)
`config/proyectos.json`	Registro de proyectos prácticos ingestados y su huella de archivos (`total_archivos`, `ultimo_mtime`)	`{"proyectos": [{"id": str, "nombre": str, "materia": str, "parcial": str, "tipo": str, "ruta": str, "nota_obsidian": str, "huella": {...}, "ultimo_analisis": str}]}`	`JsonProyectosRepo` (`infrastructure/persistence/json_repositorios.py:429-479`)

config/plugins_config.json	Vínculo manual y carpeta por plugin (cisco, nodered, web)	{"cisco": {"materia": str, "carpeta": str}, ...}	JsonPluginConfigRepo (infrastructure/persistence/json_repositorios.py:402-426)
plugins/<id>_context.md	Memoria técnica en Markdown extraída por cada plugin e inyectada en los prompts de tutoría e ingesta	Texto Markdown libre con cabeceras de fecha y análisis	PluginPracticoBase (core/ports/plugin_practico.py:22-28), PluginsService (core/application/plugins.py:44-284)
repaso/tarjetas.json	Colección de tarjetas de repaso espaciado con sus métricas SM-2	{"<id>": {"id": str, "materia": str, "tema": str, "enunciado": str, "opciones": [...], "respuesta_correcta": str, "explicacion": str, "intervalo_dias": int, "facilidad": float, "racha": int, "proximo_repaso": str, "historial": [...]}}	JsonTarjetasRepo (infrastructure/persistence/json_repaso.py:20-47)
repaso/historial.json	Registro cronológico de eventos de estudio (quiz, repaso, tutoria, guia)	[{"fecha": str, "tipo": str, "materia": str, "tema": str, ...}]	JsonHistorialRepo (infrastructure/persistence/json_repaso.py:50-87)
guias/guias.json	Guías de estudio offline creadas, con el contenido Markdown generado para cada clase y su estado de revisión	{"<id>": {"id": str, "materia": str, "creada": str, "extension": str, "tiempo_minutos": int|null, "revisada": bool, "fecha_revision": str|null, "clases": [...]}}	JsonGuiasRepo (infrastructure/persistence/json_guias.py:12-43)

3.3. Estado volátil (en memoria del servidor y navegador)

- **`SessionStore` en el backend (`apps/api/session_store.py:10-33`):** mantiene en diccionarios en RAM (`_sesiones` y `_quizzes`) las sesiones de tutoría activas y los cuestionarios recién generados (incluyendo las respuestas correctas que se ocultan al cliente en `apps/api/routers/quiz.py:15-28`). Si el proceso de `ml-web` se reinicia, las tutorías o quizzes en curso se pierden y deben volver a iniciarse.
- **`localStorage` en el navegador:**
  - `ml-tema`: tema visual (`"oscuro"` o `"claro"`, `frontend/src/lib/theme.ts:3-48`).
  - `ml:revision-guia:<id>`: borradores de notas y estados (`entendida`/`con_dudas`/`pendiente`) mientras el estudiante revisa una guía offline antes de confirmarla (`frontend/src/views/RevisionGuiaView.tsx:17, 136-148, 161`).
  - `ml_font_size`: tamaño de fuente del lector de tutoría (`frontend/src/views/TutoriaView.tsx:76-82`).

3.4. Qué escribe o consume fuera de `AlmacenDocumentos`

1. **Bóveda de Obsidian (`BovedaObsidianAdapter`, `infrastructure/obsidian/boveda_obsidian.py:12-136`):**
   - Escribe archivos `.md` dentro de la ruta `obsidian_vault` al ingestar materiales (`guardar_nota`), analizar proyectos (`guardar_nota`) o finalizar una tutoría (`guardar_resumen` + actualización de `<materia>.md`).
   - Elimina archivos `.md` al desvincular un material (`eliminar_material`), eliminar un proyecto (`eliminar_proyecto`) o vaciar materiales (`vaciar_materiales`), verificando mediante `os.path.realpath` que el archivo esté estrictamente dentro de la bóveda y que su nombre termine en `_Notas.md` o esté bajo la carpeta `Proyectos` (`boveda_obsidian.py:122-136`).
2. **Archivos temporales en el servidor:**
   - Al extraer un sílabo (`apps/api/routers/silabo.py:31-49`) o subir un archivo de material suelto (`apps/api/routers/material.py:105-121`), el endpoint crea un directorio temporal con `tempfile.mkdtemp()` / `TemporaryDirectory()`, guarda el archivo subido, lo procesa y lo elimina al finalizar.
3. **API externa de Google Gemini (`LLM_GeminiAdapter`, `infrastructure/llm/gemini_adapter.py:23-267`):**
   - Consume cuota de la API de Gemini usando la clave configurada en `config/ajustes.json` o en la variable de entorno `GEMINI_API_KEY` (`settings.py:26-33`).
   - Implementa rotación de modelos de respaldo y hasta 3 reintentos automáticos con espera progresiva (`15 * (intento + 1)` segundos) cuando recibe errores de cuota (`429` / `RESOURCE_EXHAUSTED`, `gemini_adapter.py:147-155`).

---

4. Casos de uso del sistema

A continuación se especifican los **20 casos de uso (`CU-01` a `CU-20`)** que cubren la totalidad de las vistas web del menú de 5 entradas, endpoints HTTP, comandos CLI y scripts del sistema.

CU-01: Gestionar asignaturas del catálogo (listar, crear, renombrar y eliminar)

- **Actor:** Estudiante.
- **Precondiciones:** Servidor `ml-web` en ejecución.
- **Flujo principal:**
  1. **Consultar catálogo en selector único:** al abrir «Asignaturas» (`/asignaturas`, `frontend/src/views/AsignaturasView.tsx:22-294`), `cargarMaterias` (`AsignaturasView.tsx:47-90`) se ejecuta una única vez al montar la vista (sin volver a pedir `/api/materias` al cambiar entre las pestañas `Clases`, `Material` y `Proyectos`, `P29` — commit `670143a`). Invoca `api.getMaterias()` (`frontend/src/api/client.ts:76-79`) → `GET /api/materias` (`backend/src/modular_learning/apps/api/routers/materias.py:22-29`) → `MateriasService.listar_materias()` y `listar_catalogo()` (`core/application/materias.py:35-41`) → `JsonMateriaRepo.listar()` (`infrastructure/persistence/json_repositorios.py:46-63`). Si la URL incluía `?materia=<slug_o_nombre>`, selecciona esa asignatura mediante `buscarMateriaPorClave` (`frontend/src/lib/asignaturas.ts:106-112`).
  2. **Crear asignatura:** en la barra superior de `AsignaturasView`, el estudiante pulsa «Nueva asignatura», escribe el nombre y pulsa «Guardar» (`handleCrearMateria`, `AsignaturasView.tsx:104-118, 211-242`). Se llama a `api.createMateria(limpia)` (`client.ts:81-88`) → `POST /api/materias` (`routers/materias.py:32-38`) → `MateriasService.agregar_materia(nombre)` (`core/application/materias.py:47-50`) → `JsonMateriaRepo.agregar_materia(nombre)` (`json_repositorios.py:87-95`), que crea `Materia.desde_nombre(nombre)` y la añade a `config/materias.json` solo si no existe ya una asignatura con el mismo `slug` (`P14`, commit `a9b71aa`).
  3. **Renombrar asignatura:** el estudiante pulsa el icono de lápiz junto al selector de asignatura activa, edita el nombre y confirma (`handleRenombrar`, `AsignaturasView.tsx:120-136, 171-204`). Se invoca `api.renombrarMateria(materiaActiva, limpia)` (`client.ts:90-97`) → `PUT /api/materias/{slug}` (`routers/materias.py:41-51`) → `MateriasService.renombrar_materia(slug, nuevo_nombre)` (`core/application/materias.py:52-59`), que actualiza el `nombre` visible conservando el `slug` en `JsonMateriaRepo.guardar()` (`json_repositorios.py:65-73`).
  4. **Eliminar asignatura (Zona peligrosa):** al final de la pestaña **Asignaturas › Clases**, el estudiante despliega «Zona peligrosa» (`frontend/src/views/SilabosView.tsx:993-1020`), pulsa «Eliminar asignatura» y confirma en el diálogo (`handleEliminarMateria`, `SilabosView.tsx:236-260`). Se invoca `api.eliminarMateria(materiaActiva)` (`client.ts:99-104`) → `DELETE /api/materias/{slug}` (`routers/materias.py:54-62`) → `MateriasService.eliminar_materia(slug, borrar_cronograma=True)` (`core/application/materias.py:61-80`, completado en `P28` — commit `58bf149`), que ejecuta cuatro borrados coordinados:
     - Elimina la asignatura de `config/materias.json` (`JsonMateriaRepo.eliminar`, `json_repositorios.py:75-82`).
     - Elimina `schedules/<slug>_cronograma.json` (`JsonCronogramaRepo.eliminar_cronograma`, `json_repositorios.py:191-195`).
     - Elimina todos los proyectos de esa asignatura (`ProyectosService.eliminar_proyecto`, `core/application/proyectos.py:55-64`), borrando sus fichas `.md` en Obsidian y sus entradas `[PROYECTO] ...` en `config/registro_materiales.json`.
     - Elimina todas las entradas de esa asignatura en `config/registro_materiales.json` (`JsonRegistroMaterialesRepo.eliminar_de_materia`, `json_repositorios.py:387-393`).
- **Flujos alternativos y errores:**
  - Si al crear o renombrar el nombre está vacío, `MateriasService` lanza `ValueError` y el router responde `HTTP 400` (`routers/materias.py:37, 48`).
  - Si al renombrar o eliminar el `slug` no existe en `config/materias.json`, el servicio devuelve `None` / `False` y el router responde `HTTP 404` (`routers/materias.py:50, 60`).
  - **Discrepancias asociadas: DISC-03 (P14)** (resuelta en `a9b71aa`) y **DISC-11 (P28)** (resuelta en `58bf149`: `eliminar_materia` borra también los proyectos asociados).
- **Datos que lee y escribe:**
  - Lee y escribe: `config/materias.json`, `schedules/<slug>_cronograma.json`, `config/registro_materiales.json`, `config/proyectos.json` y fichas de proyecto en `obsidian_vault`.
- **Resultado esperado:** Catálogo de asignaturas administrado desde un único selector compartido; al eliminar una asignatura desaparecen su cronograma, sus materiales registrados y sus proyectos.
- **Dónde está en el código y en la interfaz:**
  - Interfaz web: vista «Asignaturas» (`frontend/src/views/AsignaturasView.tsx:22-294`) y bloque «Zona peligrosa» en `frontend/src/views/SilabosView.tsx:236-260, 993-1020`.
  - Endpoints: `GET /api/materias`, `POST /api/materias`, `PUT /api/materias/{slug}`, `DELETE /api/materias/{slug}` (`backend/src/modular_learning/apps/api/routers/materias.py:22-62`).
  - Servicio y repositorio: `MateriasService` (`core/application/materias.py:17-80`), `JsonMateriaRepo` (`infrastructure/persistence/json_repositorios.py:29-95`).

CU-02: Subir, extraer e importar un sílabo (Web y CLI `ml es`)

- **Actor:** Estudiante.

- **Precondiciones:** Disponer de un archivo de sílabo en formato `.pdf`, `.md` o `.txt` (`EXTENSIONES_SILABO` en `core/application/silabo.py:13`) y tener configurada una clave válida de Gemini.
- **Flujo principal (Interfaz Web):**
  1. En **Asignaturas › Clases** (`/asignaturas`, `frontend/src/views/SilabosView.tsx:644-650, 1025-1192`), el estudiante pulsa «Subir sílabo», elige una asignatura en el desplegable `<select>` del catálogo (o la opción «+ Nueva asignatura…», `P15` — commit `1384372`, líneas `428-435, 1068-1091`) y selecciona o arrastra el archivo (`validateAndSetFile`, líneas `443-461`). La función pura `resolverMateriaSubida` (`frontend/src/lib/asignaturas.ts:118-143`, `P23` — commit `2f8fe52`) cruza el nombre derivado del archivo con el catálogo: si coincide con una asignatura existente, la selecciona; si no coincide pero el usuario ya tenía una asignatura elegida en el selector, conserva su elección sin pisarla.
  2. Al pulsar «Extraer clases» (`handleExtraer`, `SilabosView.tsx:463-491`), el cliente envía un `FormData` mediante `api.extraerSilabo(file, subidaMateria)` (`frontend/src/api/client.ts:194-205`) → `POST /api/silabo/extraer` (`backend/src/modular_learning/apps/api/routers/silabo.py:23-50`).
  3. El endpoint verifica que la extensión esté en `EXTENSIONES_SILABO` (`routers/silabo.py:28-29`), guarda el archivo en un directorio temporal (`líneas 31-35`) y si no se indicó nombre de materia llama a `SilaboService.sugerir_nombre_materia(nombre_limpio)` (`core/application/silabo.py:73-77`).
  4. `SilaboService.extraer_cronograma(ruta_temp, nombre_materia)` (`core/application/silabo.py:98-118`) extrae el texto con `LectorDocumentosLocal.extraer_texto(ruta_temp)` (`infrastructure/documents/lector_documentos.py:40-90`), recorta la sección temática (`delimitar_seccion_temas`, `silabo.py:35-71`) y solicita a Gemini un JSON estructurado mediante `LLM_GeminiAdapter.generar_json_estructurado` (`infrastructure/llm/gemini_adapter.py:201-227`).
  5. El endpoint devuelve la vista previa (`datos` y `materia`) sin guardarla aún en disco (`routers/silabo.py:42-47`). El directorio temporal se borra en el bloque `finally` (`línea 49`).
  6. El estudiante revisa la tabla «Revisa las clases extraídas» en `SilabosView.tsx:1118-1190` y pulsa «Guardar clases» (`handleGuardarSilaboExtraido`, líneas `493-523`), lo que invoca `api.guardarSilabo(subidaMateria.trim(), clasesExtraidas)` (`client.ts:207-217`) → `POST /api/silabo/guardar` (`routers/silabo.py:53-67`).
  7. `SilaboService.guardar_cronograma(nombre_materia, datos)` (`core/application/silabo.py:120-129`) resuelve primero el nombre canónico contra el catálogo con `resolver_materia(catalogo, nombre_materia)` (`P14`, commit `a9b71aa`), persiste el cronograma en `schedules/<slug>_cronograma.json` (`JsonCronogramaRepo.guardar_cronograma`, `json_repositorios.py:172-189`, asignando `id` a cada clase y `estado: "pendiente"`) y asegura el alta en el catálogo con `JsonMateriaRepo.agregar_materia(nombre)`. Finalmente, `SilabosView.tsx:512-514` recarga el catálogo dejando seleccionada la asignatura recién guardada aunque lleve tildes (`P12`, commit `bb1a30f`).
- **Flujo alternativo (CLI `ml es`):**
  1. El estudiante ejecuta `ml es` (`backend/src/modular_learning/apps/cli/main.py:28-33` → `apps/cli/commands/es.py:12-83`).
  2. La CLI solicita una ruta (o usa `silabos_dir` por defecto, `es.py:64-78`). Si es una carpeta, lista los archivos compatibles con `escanear_directorio_silabos` (`silabo.py:148-156`).
  3. Por cada archivo, sugiere un nombre limpio con `sugerir_nombre_materia`, permite confirmarlo o cambiarlo por consola (`es.py:44-47`), llama a `extraer_cronograma` y luego directamente a `guardar_cronograma` (`es.py:52-61`).
- **Errores:**
  - Extensión no soportada → `HTTP 400` (`routers/silabo.py:29`).
  - Si el archivo no contiene texto extraíble o falla Gemini → `ValueError` en `SilaboService.extraer_cronograma` (`silabo.py:102, 114-116`) → `HTTP 400` (`routers/silabo.py:41`).
  - **Discrepancias asociadas: DISC-01 (P12)** (`bb1a30f`), **DISC-03 (P14)** (`a9b71aa`) y **DISC-04 (P15 / P23)** (`1384372` y `2f8fe52`), todas resueltas.
- **Datos que lee y escribe:**
  - Lee: archivo de sílabo subido o desde `silabos_dir`, `config/materias.json`.
  - Escribe: `schedules/<slug>_cronograma.json` y `config/materias.json`. Consume cuota de Gemini.
- **Resultado esperado:** Cronograma estructurado guardado para la asignatura y visible inmediatamente en **Asignaturas › Clases**.
- **Dónde está en el código y en la interfaz:**
  - Interfaz web: `frontend/src/views/SilabosView.tsx:428-523, 1025-1192`, `frontend/src/lib/asignaturas.ts:118-143`.
  - CLI: `ml es` (`backend/src/modular_learning/apps/cli/commands/es.py:12-83`).
  - Endpoints: `POST /api/silabo/extraer`, `POST /api/silabo/guardar` (`apps/api/routers/silabo.py:23-67`).
  - Servicio: `SilaboService` (`core/application/silabo.py:16-156`).

CU-03: Editar el cronograma de clases, estados, cortes de parcial y consultar el Calendario

- **Actor:** Estudiante.
- **Precondiciones:** Existir al menos una asignatura en el catálogo.
- **Flujo principal:**
  1. **Cargar cronograma y parciales:** al elegir una asignatura en el selector único de `AsignaturasView` (`frontend/src/views/SilabosView.tsx:170-198`), se llama en paralelo a:

- `api.getCronogramaMateria(materiaActiva)` (`client.ts:116-119`) → `GET /api/materias/{materia}/cronograma` (`apps/api/routers/materias.py:72-78`) → `MateriasService.obtener_cronograma` (`core/application/materias.py:82-85`) → `JsonCronogramaRepo.obtener_cronograma` (`json_repositorios.py:156-170`).
- `api.getParciales(materiaActiva)` (`client.ts:173-176`) → `GET /api/materias/{materia}/parciales` (`routers/materias.py:154-159`) → `MateriasService.obtener_parciales` (`core/application/materias.py:158-162`) → `JsonCronogramaRepo.obtener_parciales` (`json_repositorios.py:307-315`).

2. **Añadir una clase manualmente:** el estudiante pulsa «+ Añadir clase», completa `tema`, `fecha`, `objetivo` y `contenido` y guarda (`handleAgregarNuevaClase`, `SilabosView.tsx:380-407, 787-821`). Se invoca `api.agregarClase(materiaActiva, payload)` (`client.ts:128-135`) → `POST /api/materias/{materia}/clases` (`routers/materias.py:100-110`) → `MateriasService.agregar_clase` (`core/application/materias.py:108-120`), que genera un `nuevo_id_clase()`, fija `estado: "pendiente"`, añade la clase al cronograma y asegura que la materia exista en `JsonMateriaRepo`.
3. **Editar una clase existente:** el estudiante pulsa el icono de editar en una fila del cronograma, modifica los campos y guarda (`handleGuardarEdicionClase`, `SilabosView.tsx:355-378, 845-880`). Se invoca `api.editarClase(materiaActiva, editandoClaseId, payload)` (`client.ts:137-151`) → `PUT /api/materias/{materia}/clases/{clase_id}` (`routers/materias.py:113-126`) → `MateriasService.editar_clase` (`core/application/materias.py:122-136`), que localiza la clase por `clase["id"] == clase_id` y actualiza sus campos.
4. **Alternar estado (`pendiente` / `completado`):** el estudiante pulsa el botón de estado de una clase (`handleToggleEstadoClase`, `SilabosView.tsx:321-339, 945-961`). Se invoca `api.cambiarEstadoClase(materiaActiva, clase.id, nuevoEstado)` (`client.ts:163-171`) → `PATCH /api/materias/{materia}/clases/{clase_id}/estado` (`routers/materias.py:141-151`) → `MateriasService.cambiar_estado_clase` (`core/application/materias.py:151-156`) → `JsonCronogramaRepo.marcar_clase` (`json_repositorios.py:232-264`). Si pasa a `"completado"`, graba `fecha_completado` con la fecha actual; si vuelve a `"pendiente"`, elimina `fecha_completado`.
5. **Eliminar una clase individual:** el estudiante pulsa el icono de papelera en la fila de la clase y confirma (`handleEliminarClase`, `SilabosView.tsx:341-353, 971-980`). Se invoca `api.eliminarClase(materiaActiva, clase.id)` (`client.ts:153-161`) → `DELETE /api/materias/{materia}/clases/{clase_id}` (`routers/materias.py:129-138`) → `MateriasService.eliminar_clase` (`core/application/materias.py:138-149`).
6. **Definir o quitar cortes de parcial asociados a una clase (`P27`, commit `2cead9e`):** en la sección «Cortes de parcial» (`SilabosView.tsx:699-785`), el estudiante escribe el nombre del corte (ej. `"Parcial 1"`), elige en el `<select>` la última clase del cronograma que entra en ese parcial (`líneas 745-759`, guardando `c.id ? String(c.id) : String(idx + 1)` en `hasta`) y pulsa «Guardar» (`handleGuardarParcial`, `líneas 286-307`). Se invoca `api.guardarParciales(materiaActiva, actualizados)` (`client.ts:178-185`) → `POST /api/materias/{materia}/parciales` (`routers/materias.py:162-174`) → `MateriasService.guardar_parciales` (`core/application/materias.py:164-173`) → `JsonCronogramaRepo.guardar_parciales` (`json_repositorios.py:317-329`). Cada píldora muestra `describirCorte(clasesCronograma, p.hasta)` (`frontend/src/lib/parciales.ts:54-67`).
7. **Vista mensual «Calendario» y próximas clases:**
   - En la entrada **Calendario** del menú lateral (`/calendario`, `frontend/src/views/CalendarioView.tsx:57-241`, rediseñada en `P21` — commit `14da5de`), se invoca `Promise.all([api.getCronogramas(), api.getMaterias()])` (`CalendarioView.tsx:64`) → `GET /api/materias/cronogramas` (`routers/materias.py:65-69`) para renderizar una cuadrícula mensual de 7 columnas con navegación entre meses (`Mes anterior`, `Hoy`, `Mes siguiente`) y un panel de detalle del día seleccionado con enlace directo a la tutoría (`enlaceTutoria` con `c.id`).
   - En `TutoriaView.tsx:116` se invoca `api.getProximasClases(materia)` (`client.ts:106-109`) → `GET /api/materias/{materia}/clases` (`routers/materias.py:92-97`) → `MateriasService.obtener_proximas_clases(materia)` (`core/application/materias.py:87-91`) para poblar el selector de clases pendientes.

- **Flujos alternativos y errores:**
  - Si `clase_id` no existe al editar, borrar o cambiar estado → `HTTP 404` (`routers/materias.py:123, 136, 149`).
  - Si el estado enviado en `PATCH` no es `"pendiente"` ni `"completado"` → `ValueError` en `MateriasService.cambiar_estado_clase` (`materias.py:152-153`) → `HTTP 400`.
  - **Discrepancia asociada: DISC-10 (P27)** (resuelta en `2cead9e`: los cortes de parcial guardan el `id` de la clase límite del cronograma y coinciden con `GuiaView`).
- **Datos que lee y escribe:**
  - Lee y escribe: `schedules/<slug>_cronograma.json` y `config/materias.json`.
- **Resultado esperado:** Clases añadidas, editadas, marcadas o eliminadas inmediatamente en el cronograma de la asignatura; cortes de parcial vinculados a su clase límite y calendario mensual interactivo.
- **Dónde está en el código y en la interfaz:**
  - Interfaz web: `frontend/src/views/SilabosView.tsx:170-407, 699-988`, `frontend/src/views/CalendarioView.tsx:57-241`, `frontend/src/lib/parciales.ts:19-67`.
  - Endpoints: `GET /api/materias/cronogramas`, `GET /api/materias/{materia}/cronograma`, `GET /api/materias/{materia}/clases`, `POST /api/materias/{materia}/clases`, `PUT /api/materias/{materia}/clases/{clase_id}`, `DELETE /api/materias/{materia}/clases/{clase_id}`, `PATCH /api/materias/{materia}/clases/{clase_id}/estado`, `GET /api/materias/{materia}/parciales`, `POST /api/materias/{materia}/parciales` (`apps/api/routers/materias.py:65-174`).

CU-04: Vaciar el cronograma de una asignatura

- **Actor:** Estudiante.
- **Precondiciones:** Tener seleccionada una asignatura con clases cargadas en **Asignaturas › Clases**.
- **Flujo principal:**
  1. Al final de **Asignaturas › Clases**, el estudiante despliega «Zona peligrosa» y pulsa el botón «Vaciar clases» (`frontend/src/views/SilabosView.tsx:993-1009`).
  2. Se abre un diálogo de confirmación que indica: *«¿Vaciar el cronograma de "{nombre}"? Se borrarán todas sus clases pero se conservará la asignatura en el catálogo.»* (`SilabosView.tsx:265-267`).
  3. Al confirmar, el frontend llama a `api.eliminarCronograma(materiaActiva)` (`frontend/src/api/client.ts:121-126`) → `DELETE /api/materias/{materia}/cronograma` (`backend/src/modular_learning/apps/api/routers/materias.py:81-89`).
  4. El endpoint invoca `servicio.eliminar_cronograma(materia, borrar_materia=False)` (`routers/materias.py:86`, corregido en `P13` — commit `bf2ac35`) → `MateriasService.eliminar_cronograma` (`core/application/materias.py:99-106`), que elimina únicamente el archivo `schedules/<slug>_cronograma.json` y conserva la asignatura en `config/materias.json`.
- **Flujos alternativos y errores:**
  - Si no existía archivo de cronograma → `HTTP 404` (`routers/materias.py:88`).
  - **Discrepancia asociada: DISC-02 (P13)** (resuelta en `bf2ac35`).
- **Datos que lee y escribe:**
  - Elimina: `schedules/<slug>_cronograma.json`. Conserva `config/materias.json`.
- **Resultado esperado:** El cronograma queda sin clases y la asignatura permanece seleccionada en el catálogo.
- **Dónde está en el código y en la interfaz:**
  - Interfaz web: `frontend/src/views/SilabosView.tsx:262-284, 993-1009`.
  - Endpoint: `DELETE /api/materias/{materia}/cronograma` (`apps/api/routers/materias.py:81-89`).
  - Servicio: `MateriasService.eliminar_cronograma` (`core/application/materias.py:99-106`).

CU-05: Escanear carpetas locales e ingestar material pendiente en lote (Web SSE y CLI `ml im`)

- **Actor:** Estudiante (en modo local o híbrido).
- **Precondiciones:** Tener configurada la ruta `carpeta_material` con subcarpetas por asignatura y la ruta `ruta_obsidian`, y no estar en modo nube (`ML_EN_NUBE != "1"`).
- **Flujo principal (Interfaz Web):**
  1. **Escaneo (omitido en la nube, `P25` — commit `7da3dab`):** en **Asignaturas › Material** (`/asignaturas?tab=material`, `frontend/src/views/MaterialView.tsx:591-745`, visible únicamente cuando `!enNube`) o desde el bloque de aviso en la portada **Hoy** (`frontend/src/views/HoyView.tsx:113-131, 342-377`, que consulta primero `api.getConfig()` y solo escanea si `!cfg.en_nube`), el estudiante pulsa «Revisar carpetas» (`handleEscanear`, `MaterialView.tsx:353-372`). Se invoca `api.escanearMaterial(materiaEscaneo)` (`client.ts:220-226`) → `GET /api/material/escanear` (`apps/api/routers/material.py:54-63`). Si `ML_EN_NUBE == "1"`, el endpoint devuelve `{"escaneos": []}` sin tocar el disco (`routers/material.py:57-58`); en modo local/híbrido invoca `IngestaService.escanear()` o `escanear_materia()` (`core/application/ingesta.py:264-312, 351-352`).
  2. `IngestaService.escanear_materia` busca la subcarpeta `<carpeta_material>/<materia>` (`ingesta.py:266`), excluye los archivos dentro de subcarpetas `PROYECTO` / `PROYECTOS` (que delega a `ProyectosService.escanear_proyectos_en_carpeta`, líneas `289-291, 305-310`), y clasifica los archivos de `EXTENSIONES_MATERIAL` (`.pdf`, `.docx`, `.txt`, `.md`, `.pptx`, `ingesta.py:29`) en `pendientes` u `omitidos` consultando `JsonRegistroMaterialesRepo.verificar_material_ingestado` (`json_repositorios.py:356-358`).
  3. **Sincronización en streaming (SSE):** cuando hay archivos pendientes, el estudiante pulsa «Convertir todo en notas» (`handleStartSync`, `MaterialView.tsx:374-385, 639`, o desde `HoyView.tsx:353-358`). El cliente abre un stream `GET`/`EventSource` o `POST /api/material/sincronizar/stream` (`apps/api/routers/material.py:66-86`) → `IngestaService.ingestar_pendientes_stream` (`core/application/ingesta.py:354-406`).
  4. Por cada archivo pendiente, el generador ejecuta `IngestaService.ingestar_archivo` (`ingesta.py:159-228`):
     - Resuelve el nombre canónico de la asignatura contra el catálogo (`_nombre_materia`, `ingesta.py:152-157, 167`, añadido en `P14` — commit `a9b71aa`).
     - Genera el contenido en Markdown (`_generar_contenido`, `ingesta.py:86-132`): si es `.pdf`, usa `llm.generar_texto_con_archivo`; si es `.pptx`, convierte primero a PDF temporal con `ConversorLibreOffice` (`infrastructure/documents/conversor_libreoffice.py:40-112`, con perfil temporal `-env:UserInstallation`) o aplica el plan B con `python-pptx` (`lector_documentos.py:56-84`, exigiendo al menos `200` caracteres útiles en `ingesta.py:114-120`, `P24` — commit `faf23ac`); si es `.docx`, `.txt` o `.md`, extrae el texto plano con `LectorDocumentosLocal.extraer_texto` y llama a `llm.generar_texto`.
     - Guarda la nota Markdown resultante en `<obsidian_vault>/<materia>/<Tipo>/<base>_Notas.md` (`BovedaObsidianAdapter.guardar_nota`, `infrastructure/obsidian/boveda_obsidian.py:28-42`).
     - Registra el archivo en `config/registro_materiales.json` (`JsonRegistroMaterialesRepo.registrar_material`,

json_repositorios.py:360-373).
    - Alinea el material con la clase correspondiente del cronograma (_alinear_con_cronograma, ingesta.py:134-150 → asignar_material_a_clase, core/domain/alineador_curricular.py:51-109).
    - Si existe un plugin activo para la asignatura, extrae el listado técnico y lo guarda con plugin.guardar_contexto_extraido (ingesta.py:201-215).
  5. Al terminar el lote, emite el evento de cierre con los totales procesados (ingesta.py:401-406).
- **Flujo alternativo (CLI `ml im`):**
  1. El estudiante ejecuta `ml im` (apps/cli/main.py:36-41 → apps/cli/commands/im.py:12-106).
  2. La CLI lista las asignaturas de MateriasService.listar_materias(), pide elegir una (o todas), escanea con ingesta_service.escanear_materia(materia) (im.py:16), muestra los archivos pendientes y los proyectos detectados, y al confirmar procesa cada archivo con ingesta_service.ingestar_archivo (im.py:31) y cada proyecto nuevo o modificado con proyectos_service.analizar_e_ingestar_proyecto (im.py:45).
- **Errores y discrepancias asociadas:**
  - **DISC-05 (P16 / P24):** Resuelta en d40428d y faf23ac.
  - **DISC-07:** Los archivos .docx solo extraen texto de párrafos (lector_documentos.py:51-55), ignorando tablas e imágenes embebidas.
  - **DISC-08 (P25):** Resuelta en 7da3dab (en modo nube se oculta el escáner y no se llaman ni ejecutan escaneos de carpetas locales).
- **Datos que lee y escribe:**
  - Lee: archivos en carpeta_material, config/materias.json, config/registro_materiales.json, schedules/<slug>_cronograma.json, plugins/<id>_context.md.
  - Escribe: notas .md en ruta_obsidian, config/registro_materiales.json, schedules/<slug>_cronograma.json y plugins/<id>_context.md.
- **Resultado esperado:** Todos los materiales pendientes quedan convertidos en notas de Obsidian, registrados y vinculados a su clase en el cronograma.
- **Dónde está en el código y en la interfaz:**
  - Interfaz web: frontend/src/views/MaterialView.tsx:313-385, 591-745, 954-995 y frontend/src/views/HoyView.tsx:113-131, 342-377.
  - CLI: `ml im` (backend/src/modular_learning/apps/cli/commands/im.py:12-106).
  - Endpoints: GET /api/material/escanear, POST /api/material/sincronizar/stream (apps/api/routers/material.py:54-86).
  - Servicio: IngestaService (core/application/ingesta.py:69-406).

CU-06: Subir e ingestar un archivo de material suelto

- **Actor:** Estudiante (en modo local, nube o híbrido).
- **Precondiciones:** Tener un archivo .pdf, .docx, .txt, .md o .pptx en el dispositivo desde el que se abre el navegador.
- **Flujo principal:**
  1. En **Asignaturas › Material** (/asignaturas?tab=material), el estudiante pulsa el botón «Subir un archivo» en la cabecera de «Materiales convertidos en notas» (mostrarSubida, plegada por defecto desde P17 — commit 2d6f757, frontend/src/views/MaterialView.tsx:762-770, 837-938).
  2. La asignatura se toma automáticamente del selector único de AsignaturasView (materiaFiltro, o permite elegirla en el <select> si no está filtrada, MaterialView.tsx:850-869, P15 — commit 1384372), selecciona el tipo de material y elige el archivo con accept=".pdf,.docx,.pptx,.txt,.md" (línea 891, P24 — commit faf23ac).
  3. Al pulsar «Convertir en nota de Obsidian» (handleUploadIndividual, MaterialView.tsx:387-416), el cliente invoca api.ingestarArchivo(uploadFile, uploadMateria, uploadTipo) (frontend/src/api/client.ts:243-258) → POST /api/material/ingestar (backend/src/modular_learning/apps/api/routers/material.py:89-121).
  4. El endpoint comprueba que la extensión esté en EXTENSIONES_MATERIAL (.pdf, .docx, .txt, .md, .pptx, routers/material.py:99-103).
  5. Llama a MateriasService.agregar_materia(materia_limpia) (routers/material.py:108, que no sobrescribe si el slug ya existe), guarda el archivo en un tempfile.TemporaryDirectory() (líneas 110-113) y ejecuta IngestaService.ingestar_archivo(ruta, nombre, materia_limpia, tipo) (línea 115), el cual resuelve el nombre canónico en el catálogo (_nombre_materia, ingesta.py:167).
  6. Al salir del bloque with tempfile.TemporaryDirectory(), borra el directorio temporal y devuelve la ruta de la nota generada, la alineación curricular y, si se usó el plan B de .pptx, el campo aviso (mostrado en el navegador mediante toast.warning en MaterialView.tsx:396-403).
- **Flujos alternativos y errores:**
  - Extensión fuera de EXTENSIONES_MATERIAL, presentación .pptx de solo imágenes sin LibreOffice o asignatura vacía → HTTP 400

(routers/material.py:100–119).

- ○ **Discrepancias asociadas: DISC-03 (P14)** (a9b71aa), **DISC-04 (P15)** (1384372), **DISC-05 (P16 / P24)** (d40428d y faf23ac) y **DISC-06 (P17)** (2d6f757), todas resueltas.

- **Datos que lee y escribe:**
  - ○ Lee: archivo subido por multipart, config/materias.json, schedules/<slug>_cronograma.json, plugins/<id>_context.md.
  - ○ Escribe: config/materias.json, nota .md en ruta_obsidian/<materia>/<Tipo>/, config/registro_materiales.json y schedules/<slug>_cronograma.json.
- **Resultado esperado:** El archivo suelto queda procesado en Obsidian bajo el nombre canónico de la asignatura, registrado en el historial y alineado con su clase del cronograma.
- **Dónde está en el código y en la interfaz:**
  - ○ Interfaz web: frontend/src/views/MaterialView.tsx:387–416, 762–770, 837–938.
  - ○ Endpoint: POST /api/material/ingestar (apps/api/routers/material.py:89–121).
  - ○ Servicio: IngestaService.ingestar_archivo (core/application/ingesta.py:159–228).

CU-07: Revisar historial de materiales ingestados y desvincular un material

- **Actor:** Estudiante.
- **Precondiciones:** Existir materiales previamente ingestados en config/registro_materiales.json.
- **Flujo principal:**
  1. Al abrir **Asignaturas › Material** (/asignaturas?tab=material), cargarRegistrados() (frontend/src/views/MaterialView.tsx:181–190) llama a api.getMaterialesRegistrados() (client.ts:228–231) → GET /api/material/registrados (apps/api/routers/material.py:26–30) → IngestaService.listar_materiales_ingestados() (core/application/ingesta.py:230–232) → JsonRegistroMaterialesRepo.listar_materiales() (json_repositorios.py:375–376).
  2. La tarjeta «Materiales convertidos en notas» se muestra en MaterialView.tsx:748–834 (reorganizado en P17 — commit 2d6f757), filtrada por la asignatura activa en AsignaturasView y mostrando cada archivo con su asignatura, tipo y ruta de nota de Obsidian.
  3. Cuando el estudiante pulsa el botón de papelera junto a un material y confirma en el diálogo (handleEliminarMaterial, MaterialView.tsx:418–440), se invoca api.eliminarMaterial(archivo) (client.ts:233–241) → DELETE /api/material/{nombre_archivo:path} (apps/api/routers/material.py:42–51) → IngestaService.eliminar_material(nombre_archivo) (core/application/ingesta.py:234–262).
  4. IngestaService.eliminar_material realiza tres acciones:
     - ○ Elimina la entrada de config/registro_materiales.json (JsonRegistroMaterialesRepo.eliminar_material, json_repositorios.py:378–385).
     - ○ Si tenía nota, elimina la nota .md de la bóveda (BovedaObsidianAdapter.eliminar_nota, boveda_obsidian.py:122–136).
     - ○ Si tenía materia y existe cronograma, quita el archivo de material_transversal y de materiales_asociados de todas las clases del cronograma (ingesta.py:248–260).
- **Flujos alternativos y errores:**
  - ○ Si nombre_archivo no existe en config/registro_materiales.json → HTTP 404 (routers/material.py:49).
  - ○ **Discrepancia asociada: DISC-06 (P17)** (resuelta en 2d6f757).
- **Datos que lee y escribe:**
  - ○ Lee y escribe: config/registro_materiales.json, schedules/<slug>_cronograma.json, y elimina la nota correspondiente en ruta_obsidian.
- **Resultado esperado:** El material desaparece del historial, se borra su nota de Obsidian y se desvincula de las clases del cronograma.
- **Dónde está en el código y en la interfaz:**
  - ○ Interfaz web: frontend/src/views/MaterialView.tsx:181–190, 418–440, 748–834.
  - ○ Endpoints: GET /api/material/registrados, DELETE /api/material/{nombre_archivo:path} (apps/api/routers/material.py:26–30, 42–51).
  - ○ Servicio: IngestaService.listar_materiales_ingestados y eliminar_material (core/application/ingesta.py:230–262).

CU-08: Escanear, ingestar, verificar cambios, actualizar y desvincular proyectos de asignatura

- **Actor:** Estudiante.
- **Precondiciones:** En modo local/híbrido, contar con carpetas de proyecto dentro de <carpeta_material>/<Asignatura>/Proyectos/....
- **Flujo principal:**
  1. **Listar y escanear proyectos:** en la pestaña **Asignaturas › Proyectos** (/asignaturas?tab=proyectos, frontend/src/views/

AsignaturasView.tsx:285-291 → MaterialView.tsx:999-1250), cargarProyectos() (MaterialView.tsx:212-238) invoca api.getProyectos() (client.ts:285-289) → GET /api/proyectos (apps/api/routers/proyectos.py:23-29) y, solo cuando !enNube (MaterialView.tsx:222, P25 — commit 7da3dab), llama a api.escanearProyectos() (client.ts:291-295) → GET /api/proyectos/escanear (routers/proyectos.py:32-50). Si ML_EN_NUBE == "1", el endpoint devuelve {"candidatos": []} sin tocar el disco (routers/proyectos.py:37-38).

2. En modo local/híbrido, GET /api/proyectos/escanear recorre las asignaturas, llama a ProyectosService.escanear_proyectos_en_carpeta(ruta, nombre) (core/application/proyectos.py:66-74) → InspectorProyectosLocal.escanear (infrastructure/proyectos/inspector_local.py:83-99) y devuelve cada carpeta detectada.
3. **Ingestar un proyecto detectado:** en la tarjeta «Proyectos detectados en carpeta» (MaterialView.tsx:1018-1109), el estudiante pulsa «Convertir en nota» sobre un proyecto pendiente (handleIngestarProyecto, líneas 442-463), llamando a api.ingestarProyecto(...) (client.ts:302-309) → POST /api/proyectos/ingestar (routers/proyectos.py:75-90) → ProyectosService.analizar_e_ingestar_proyecto (core/application/proyectos.py:126-167):
   - Calcula HuellaProyecto (inspector_local.py:101-127), detecta el tipo (software, iot, documental_auditoria, documento_rector) y extrae el contexto técnico de los archivos del proyecto (inspector_local.py:149-185).
   - Genera una ficha técnica en Markdown con Gemini usando un prompt especializado según el tipo de proyecto (generar_ficha_markdown, proyectos.py:76-124).
   - Guarda la ficha en <ruta_obsidian>/<materia>/Proyectos/<nombre>.md (proyectos.py:149-153).
   - Persiste el objeto Proyecto en config/proyectos.json (JsonProyectosRepo.guardar, json_repositorios.py:435-440) y añade también una entrada [PROYECTO] {nombre} ({etiqueta_parcial}) en config/registro_materiales.json (proyectos.py:162-166).
4. **Verificar estado y actualizar un proyecto modificado:** en la tarjeta «Fichas de proyectos registradas» (MaterialView.tsx:1111-1250), verificarEstados (líneas 192-210) consulta GET /api/proyectos/{id_proyecto:path}/estado (routers/proyectos.py:53-61). Si detecta cambios en los archivos del proyecto, la fila muestra el botón «Actualizar» (handleActualizarProyecto, MaterialView.tsx:465-480), que llama a api.actualizarProyecto(id) (client.ts:311-316) → POST /api/proyectos/{id_proyecto:path}/actualizar (routers/proyectos.py:93-104) → ProyectosService.actualizar_proyecto (proyectos.py:196-207), re-ejecutando el análisis y actualizando la nota y la huella. También existe GET /api/proyectos/{id_proyecto:path} (routers/proyectos.py:64-72) para consultar un proyecto por su id.
5. **Desvincular un proyecto (P28, commit 58bf149):** el estudiante pulsa el icono de papelera en un proyecto registrado (handleEliminarProyecto, MaterialView.tsx:482-500), invocando api.eliminarProyecto(id) (client.ts:318-323) → DELETE /api/proyectos/{id_proyecto:path} (routers/proyectos.py:107-117) → ProyectosService.eliminar_proyecto(id_proyecto, borrar_nota=True) (proyectos.py:55-64), que borra el proyecto de config/proyectos.json, elimina su entrada [PROYECTO] {nombre} ({etiqueta_parcial}) de config/registro_materiales.json y elimina su ficha .md en Obsidian.

- **Flujos alternativos y errores:**
  - Proyecto inexistente o carpeta borrada al actualizar → HTTP 400 / HTTP 404 (routers/proyectos.py:70, 88, 102, 113).
  - **Discrepancias asociadas: DISC-08 (P25)** (resuelta en 7da3dab) y **DISC-11 (P28)** (resuelta en 58bf149: eliminar_proyecto limpia también registro_materiales.json y la nota en Obsidian).
- **Datos que lee y escribe:**
  - Lee: carpetas bajo <carpeta_material>/<Asignatura>/Proyectos, config/proyectos.json.
  - Escribe: config/proyectos.json, config/registro_materiales.json, <ruta_obsidian>/<materia>/Proyectos/<nombre>.md.
- **Resultado esperado:** Proyectos analizados como fichas técnicas en Obsidian dentro de su propia pestaña **Asignaturas › Proyectos**, con detección de cambios por huella y borrado consistente.
- **Dónde está en el código y en la interfaz:**
  - Interfaz web: frontend/src/views/AsignaturasView.tsx:285-291, frontend/src/views/MaterialView.tsx:192-238, 442-500, 999-1250.
  - CLI: integrado dentro de ml im (apps/cli/commands/im.py:21-25, 42-50).
  - Endpoints: GET /api/proyectos, GET /api/proyectos/escanear, GET /api/proyectos/{id_proyecto:path}/estado, GET /api/proyectos/{id_proyecto:path}, POST /api/proyectos/ingestar, POST /api/proyectos/{id_proyecto:path}/actualizar, DELETE /api/proyectos/{id_proyecto:path} (apps/api/routers/proyectos.py:23-117).
  - Servicio e inspector: ProyectosService (core/application/proyectos.py:29-218), InspectorProyectosLocal (infrastructure/proyectos/inspector_local.py:55-186).

CU-09: Vaciar todos los materiales y proyectos ingestados

- **Actor:** Estudiante.
- **Precondiciones:** Existir materiales registrados en config/registro_materiales.json o proyectos en config/proyectos.json.
- **Flujo principal:**
  1. Al final de **Asignaturas › Material**, el estudiante despliega «Zona peligrosa» (frontend/src/views/

MaterialView.tsx:1254-1276), pulsa «Vaciar materiales y proyectos» y confirma en el diálogo (handleVaciarTodo, líneas 502-527).

2. El cliente invoca api.vaciarMateriales(true) (frontend/src/api/client.ts:325-332) → POST /api/material/vaciar (backend/src/modular_learning/apps/api/routers/material.py:33-39).
3. El endpoint ejecuta en secuencia (P28, commit 58bf149):
   - IngestaService.vaciar_materiales(borrar_notas_obsidian=True) (core/application/ingesta.py:314-349): recorre config/registro_materiales.json, elimina cada nota .md en Obsidian mediante BovedaObsidianAdapter.eliminar_nota, vacía config/registro_materiales.json (JsonRegistroMaterialesRepo.vaciar, json_repositorios.py:395-399) y recorre todos los cronogramas limpiando materiales_asociados en cada clase y material_transversal en la asignatura (JsonCronogramaRepo.limpiar_material_transversal, json_repositorios.py:298-305).
   - ProyectosService.vaciar_proyectos(borrar_notas_obsidian=True) (core/application/proyectos.py:209-218): elimina las fichas de proyectos en Obsidian y vacía config/proyectos.json (JsonProyectosRepo.vaciar, json_repositorios.py:466-471).

- **Flujos alternativos y errores:**
  - Si alguna nota ya no existe en disco, se ignora el error y se contabilizan únicamente las notas efectivamente borradas (ingesta.py:326-329).
  - **Discrepancia asociada: DISC-11 (P28)** (resuelta en 58bf149: vaciar_materiales limpia también materiales_asociados y material_transversal en los cronogramas).
- **Datos que lee y escribe:**
  - Escribe: vacía config/registro_materiales.json y config/proyectos.json, limpia materiales_asociados y material_transversal en schedules/*_cronograma.json, y elimina las notas .md generadas en ruta_obsidian.
- **Resultado esperado:** Historial de materiales, proyectos y referencias en cronogramas completamente limpio, permitiendo re-ingestar desde cero.
- **Dónde está en el código y en la interfaz:**
  - Interfaz web: frontend/src/views/MaterialView.tsx:502-527, 1254-1276.
  - Endpoint: POST /api/material/vaciar (apps/api/routers/material.py:33-39).
  - Servicios: IngestaService.vaciar_materiales (core/application/ingesta.py:314-349) y ProyectosService.vaciar_proyectos (core/application/proyectos.py:209-218).

CU-10: Realizar una sesión de tutoría interactiva (Web y CLI `ml init`)

- **Actor:** Estudiante.
- **Precondiciones:** Tener una asignatura (y opcionalmente clases en su cronograma) y clave de Gemini activa.
- **Flujo principal (Interfaz Web):**
  1. **Configurar e iniciar sesión:** en **Estudiar** › **Tutoría** (/estudiar?tab=tutoria, frontend/src/views/EstudiarView.tsx:16-71 → frontend/src/views/TutoriaView.tsx:65-697, a la que también se llega pre-rellenada con enlaceTutoria desde HoyView.tsx:196, 225, 290, CalendarioView.tsx:48, RevisionGuiaView.tsx:69, 87 o al fallar preguntas en QuizView.tsx:138), el estudiante elige la asignatura, selecciona una clase pendiente del cronograma (o activa el modo «Tema libre»), añade subtemas adicionales o un enfoque particular y pulsa «Empezar tutoría» (handleIniciar, TutoriaView.tsx:161-193).
  2. El cliente invoca api.iniciarTutoria(...) (frontend/src/api/client.ts:335-342) → POST /api/tutoria/iniciar (backend/src/modular_learning/apps/api/routers/tutoria.py:33-69).
  3. El endpoint registra la materia con MateriasService.agregar_materia(req.materia) (routers/tutoria.py:55, sin sobrescribir el nombre si ya existe) e invoca TutoriaService.iniciar_sesion (core/application/tutoria.py:118-172):
     - Busca la clase en el cronograma por clase_id (o por nombre de tema si no se envió clase_id) usando JsonCronogramaRepo.obtener_detalle_clase (json_repositorios.py:221-230) para recuperar sus materiales_asociados y subtemas_adicionales.
     - Resuelve el nombre canónico de la asignatura contra el catálogo con resolver_materia(self.materia_repo.listar(), materia) (tutoria.py:135-138, P14 — commit a9b71aa).
     - Pide a Gemini un roadmap JSON de pasos (01_roadmap_aprendizaje.md, tutoria.py:152-155).
     - Devuelve la instancia SesionTutoria con pasos, enfoque, fallos_previos y clase_id.
  4. El router guarda la sesión en RAM mediante SessionStore.guardar_sesion (apps/api/session_store.py:17-18) y devuelve sesion_id y ruta_aprendizaje (routers/tutoria.py:58-65).
  5. **Explicación de cada paso en streaming (SSE):** automáticamente al iniciar o al cambiar de paso (startStream, TutoriaView.tsx:127-159), el frontend abre un stream GET /api/tutoria/{sesion_id}/paso/{indice}/stream (routers/tutoria.py:72-98). TutoriaService.explicar_paso_stream (tutoria.py:232-245) construye el prompt (construir_prompt_explicacion, tutoria.py:174-219, integrando materiales del Segundo Cerebro, fallos previos y el plugin práctico de la asignatura) e invoca LLM_GeminiAdapter.generar_respuesta_stream (gemini_adapter.py:229-266), enviando fragmentos por SSE.
  6. **Resolver dudas del paso actual:** el estudiante escribe una pregunta en el chat lateral de dudas (handleEnviarDuda,

TutoriaView.tsx:195-222), que llama a api.resolverDuda(sesion.sesion_id, ...) (client.ts:344-351) → POST /api/tutoria/{sesion_id}/duda (routers/tutoria.py:101-115) → TutoriaService.responder_duda (tutoria.py:247-260).

7. **Finalizar sesión y guardar en Obsidian:** al terminar los pasos, el estudiante pulsa «Terminar y guardar en Obsidian» (handleFinalizar, TutoriaView.tsx:224-243), llamando a api.finalizarTutoria(sesion.sesion_id) (client.ts:353-358) → POST /api/tutoria/{sesion_id}/finalizar (routers/tutoria.py:118-132) → TutoriaService.finalizar_sesion (tutoria.py:262-296), que:
   - Genera la guía de estudio final con Gemini (03_resumen_evaluacion.md, tutoria.py:267-274).
   - Escribe el resumen en <ruta_obsidian>/<materia>/<tema>.md y actualiza el índice <ruta_obsidian>/<materia>.md (BovedaObsidianAdapter.guardar_resumen, boveda_obsidian.py:44-94).
   - Marca la clase como "completado" en el cronograma por clase_id (o por tema si fue tema libre) mediante JsonCronogramaRepo.marcar_clase_completada (json_repositorios.py:266-269).
   - Registra el evento de estudio y crea tarjetas de repaso mediante RepasoService.registrar_tutoria (core/application/repaso.py:89-102).
   - Elimina la sesión de SessionStore (routers/tutoria.py:124).

- **Flujo alternativo (CLI `ml init`):**
  1. El estudiante ejecuta ml init (apps/cli/main.py:20-25 → apps/cli/commands/init.py:13-95).
  2. pedir_datos_inicio (apps/cli/ui/interaccion.py:33-126, actualizado en P26 — commit 511c3ab) pide seleccionar materia, elegir una clase pendiente del cronograma o tema libre, y conserva clase_id_seleccionada = clase_seleccionada.get("id") (línea 83), devolviéndola en "clase_id" (línea 106).
  3. init.py:29-36 invoca tutoria_service.iniciar_sesion(..., clase_id=datos.get("clase_id")), transmite cada explicación en consola con paneles Rich.Live (init.py:49-56) y al finalizar con tutoria_service.finalizar_sesion (init.py:83) marca como completada exactamente esa clase por su id.
- **Errores y discrepancias asociadas:**
  - Si sesion_id no existe en SessionStore (p. ej. tras reiniciar el servidor) → HTTP 404 (routers/tutoria.py:29).
  - **DISC-03 (P14):** Resuelta en a9b71aa (iniciar_sesion resuelve materia contra el catálogo).
  - **DISC-09 (P26):** Resuelta en 511c3ab (ml init envía clase_id al iniciar sesión de tutoría).
- **Datos que lee y escribe:**
  - Lee: config/materias.json, schedules/<slug>_cronograma.json, plugins/<id>_context.md, SessionStore.
  - Escribe: <ruta_obsidian>/<materia>/<tema>.md, <ruta_obsidian>/<materia>.md, schedules/<slug>_cronograma.json, repaso/tarjetas.json, repaso/historial.json.
- **Resultado esperado:** Sesión completada, resumen guardado en Obsidian, clase exacta marcada como completada por clase_id y subtemas incorporados al sistema de repaso espaciado.
- **Dónde está en el código y en la interfaz:**
  - Interfaz web: frontend/src/views/EstudiarView.tsx:16-71, frontend/src/views/TutoriaView.tsx:65-697.
  - CLI: ml init (backend/src/modular_learning/apps/cli/commands/init.py:13-95, apps/cli/ui/interaccion.py:33-126).
  - Endpoints: POST /api/tutoria/iniciar, GET /api/tutoria/{sesion_id}/paso/{indice}/stream, POST /api/tutoria/{sesion_id}/duda, POST /api/tutoria/{sesion_id}/finalizar (apps/api/routers/tutoria.py:33-132).
  - Servicio: TutoriaService (core/application/tutoria.py:95-296).

CU-11: Generar y evaluar un quiz de recuerdo activo con IA (Web y CLI `ml quiz`)

- **Actor:** Estudiante.
- **Precondiciones:** Tener al menos una asignatura con clases completadas en su cronograma y clave de Gemini activa.
- **Flujo principal (Interfaz Web):**
  1. En **Estudiar › Repaso** (/estudiar?tab=repaso, frontend/src/views/EstudiarView.tsx:16-71 → frontend/src/views/QuizView.tsx:32-559), el estudiante pulsa «Preguntas nuevas sobre temas vistos» (handleGenerarQuiz('nuevas'), QuizView.tsx:62-86, 294, 540).
  2. El cliente invoca api.generarQuiz() (frontend/src/api/client.ts:361-364) → GET /api/quiz/generar (backend/src/modular_learning/apps/api/routers/quiz.py:31-42) → EvaluacionService.generar_quiz() (core/application/evaluacion.py:92-156):
     - Obtiene las clases con estado == "completado" de los cronogramas (obtener_temas_completados, evaluacion.py:73-90).
     - Selecciona hasta 5 temas al azar y solicita a Gemini un cuestionario de opción múltiple (04_generador_quiz.md, evaluacion.py:99-125), mezclando el orden de las opciones con mezclar_opciones (evaluacion.py:24-47).
  3. El router invoca publicar_quiz(store, preguntas) (routers/quiz.py:23-28), que guarda las preguntas completas (con respuesta_correcta y explicacion) en SessionStore.guardar_quiz (apps/api/session_store.py:26-27) y devuelve al navegador solo la copia pública (_pregunta_publica, routers/quiz.py:15-20, sin respuesta_correcta ni explicacion) junto con quiz_id.
  4. El estudiante responde cada pregunta en pantalla (QuizView.tsx:379-473) y al terminar pulsa «Terminar y corregir»

(`handleEnviarQuiz`, líneas `95-127`), llamando a `api.evaluarQuiz({ quiz_id, respuestas })` (`client.ts:366-373`) → `POST /api/quiz/evaluar` (`routers/quiz.py:45-57`).

5. El router recupera y elimina el quiz de `SessionStore.eliminar_quiz(req.quiz_id)` (`routers/quiz.py:48`), llama a `EvaluacionService.evaluar_respuestas(preguntas, req.respuestas)` (`core/application/evaluacion.py:158-188`) y registra los resultados en `RepasoService.registrar_resultados` (`core/application/repaso.py:43-87`), que crea o actualiza las tarjetas en `repaso/tarjetas.json` con el algoritmo SM-2 y añade un evento en `repaso/historial.json`.
6. En la pantalla de resultados (`QuizView.tsx:160-318`), cada pregunta fallada muestra un enlace «Repasar este tema con el tutor» (`enlaceRepaso`, `líneas 134-139, 267-275`), que abre **Estudiar › Tutoría** pre-configurada con la asignatura, el tema y el enunciado fallado como enfoque.

- **Flujo alternativo (CLI `ml quiz`):**
  1. El estudiante ejecuta `ml quiz` (`apps/cli/main.py:44-49` → `apps/cli/commands/quiz.py:14-81`).
  2. Genera el cuestionario con `evaluacion_service.generar_quiz()`, presenta cada pregunta interactivamente en la terminal, evalúa con `evaluar_respuestas`, registra en `repaso_service.registrar_resultados` y muestra la calificación y explicación de cada respuesta (`apps/cli/commands/quiz.py:28-76`).
- **Errores:**
  - Si aún no hay clases completadas en ningún cronograma → `HTTP 400` (`routers/quiz.py:39`).
  - `quiz_id` inexistente o ya calificado en `SessionStore` → `HTTP 404` (`routers/quiz.py:50`).
- **Datos que lee y escribe:**
  - Lee: `schedules/<slug>_cronograma.json`, `config/materias.json`, `SessionStore`.
  - Escribe: `repaso/tarjetas.json`, `repaso/historial.json` y estado efímero en `SessionStore`.
- **Resultado esperado:** Cuestionario calificado con retroalimentación por pregunta y tarjetas de repaso espaciado creadas o reprogramadas según aciertos y fallos.
- **Dónde está en el código y en la interfaz:**
  - Interfaz web: `frontend/src/views/EstudiarView.tsx:16-71`, `frontend/src/views/QuizView.tsx:32-559`.
  - CLI: `ml quiz` (`backend/src/modular_learning/apps/cli/commands/quiz.py:14-81`).
  - Endpoints: `GET /api/quiz/generar`, `POST /api/quiz/evaluar` (`apps/api/routers/quiz.py:15-57`).
  - Servicio: `EvaluacionService` (`core/application/evaluacion.py:50-194`).

CU-12: Practicar repaso espaciado de tarjetas vencidas, consultar historial y eliminar tarjetas

- **Actor:** Estudiante.
- **Precondiciones:** Tener tarjetas registradas en `repaso/tarjetas.json` (provenientes de quizzes previos, tutorías o guías offline).
- **Flujo principal:**
  1. **Consultar resumen de repaso:** al entrar en **Estudiar › Repaso** (`frontend/src/views/QuizView.tsx:53-60`) o en la portada **Hoy** (`frontend/src/views/HoyView.tsx:96-99, 299-330, 333-341`), el cliente invoca `api.getResumenRepaso()` (`client.ts:376-379`) → `GET /api/repaso/resumen` (`apps/api/routers/repaso.py:17-21`) → `RepasoService.resumen()` (`core/application/repaso.py:155-163`), que calcula cuántas tarjetas están vencidas (`pendientes_hoy`), cuántas están `dominadas` (`intervalo_dias >= 21`), cuántas están `con_fallo_reciente` y el total de tarjetas.
  2. **Iniciar sesión de repaso de pendientes:** el estudiante pulsa «Repasar N pendientes de hoy» (`handleGenerarQuiz('vencidas')`, `QuizView.tsx:62-86, 512`), llamando a `api.repasoPendiente()` (`client.ts:382-385`) → `GET /api/repaso/pendientes` (`routers/repaso.py:24-33`) → `RepasoService.preguntas_pendientes()` (`repaso.py:128-153`). El servicio selecciona hasta `LIMITE_DIARIO = 10` tarjetas vencidas priorizando fallos recientes y antigüedad, y el router las publica mediante `publicar_quiz` (`routers/quiz.py:23-28`).
  3. **Calificar el repaso:** al terminar las tarjetas, `QuizView` envía las respuestas al mismo endpoint `POST /api/quiz/evaluar` (CU-11, paso 4), que actualiza los intervalos SM-2 (`1 → 3 → round(intervalo * facilidad)` días si acierta, o reinicia a `1` día si falla, `core/domain/repaso_espaciado.py:74-108`).
  4. **Consultar historial de sesiones:** el cliente dispone de `api.getHistorial(materia, tema)` (`client.ts:394-400`) → `GET /api/repaso/historial` (`routers/repaso.py:36-42`) → `RepasoService.historial` (`repaso.py:173-176`) → `JsonHistorialRepo.listar` (`infrastructure/persistence/json_repaso.py:65-77`).
  5. **Eliminar una tarjeta de repaso:** el endpoint `DELETE /api/repaso/tarjetas/{tarjeta_id}` (`routers/repaso.py:45-53`) → `RepasoService.eliminar_tarjeta(tarjeta_id)` (`repaso.py:178-180`) → `JsonTarjetasRepo.eliminar` (`json_repaso.py:42-47`) borra una tarjeta específica por su hash `id` (o responde `HTTP 404` si no existe).
- **Flujos alternativos y errores:**
  - Si no hay tarjetas vencidas para hoy, `GET /api/repaso/pendientes` responde `HTTP 400` indicando que no hay preguntas pendientes para hoy (`routers/repaso.py:29-32`).
- **Datos que lee y escribe:**
  - Lee y escribe: `repaso/tarjetas.json` y `repaso/historial.json`.
- **Resultado esperado:** Tarjetas vencidas repasadas y reprogramadas con sus nuevos intervalos SM-2.

- **Dónde está en el código y en la interfaz:**
  - Interfaz web: `frontend/src/views/QuizView.tsx:53-127, 476-556` y `frontend/src/views/HoyView.tsx:96-99, 299-341`.
  - Endpoints: `GET /api/repaso/resumen`, `GET /api/repaso/pendientes`, `GET /api/repaso/historial`, `DELETE /api/repaso/tarjetas/{tarjeta_id}` (`apps/api/routers/repaso.py:17-53`).
  - Servicio y dominio: `RepasoService` (`core/application/repaso.py:29-180`), `repaso_espaciado.py` (`core/domain/repaso_espaciado.py:15-108`).

CU-13: Crear una guía de estudio offline y generar sus clases con IA

- **Actor:** Estudiante.
- **Precondiciones:** Tener una asignatura con clases cargadas en su cronograma y clave de Gemini activa.
- **Flujo principal:**
  1. **Listar guías existentes y pendientes:** al abrir **Estudiar › Guía** (`/estudiar?tab=guia`, `frontend/src/views/EstudiarView.tsx:16-71` → `frontend/src/views/GuiaView.tsx:207-612`), `cargarGuias` (`GuiaView.tsx:228-233`) invoca `api.listarGuias()` (`client.ts:403-406`) → `GET /api/guias` (`apps/api/routers/guias.py:42-44`) → `GuiaOfflineService.listar()` (`core/application/guias.py:150-151`). Asimismo, en `HoyView.tsx:100-112` se invoca `api.getGuiasPendientes()` (`client.ts:408-411`) → `GET /api/guias/pendientes` (`routers/guias.py:47-50`) → `GuiaOfflineService.pendientes_de_confirmar()` (`guias.py:153-154`) para mostrar el recordatorio de guías pendientes de confirmar.
  2. **Seleccionar clases (manualmente o por corte de parcial):** el estudiante elige una asignatura, marca una o varias clases del cronograma o pulsa uno de los botones de «Cortes de parcial» (`seleccionarCorteParcial`, `GuiaView.tsx:345-363, 438-467`), que utiliza `resolverIndiceHasta(actual.clases, p.hasta)` (`frontend/src/lib/parciales.ts:19-48`, `P27` — commit `2cead9e`) para marcar automáticamente todas las clases pendientes desde el inicio del corte hasta la clase límite del parcial. Opcionalmente indica cuántos minutos tiene disponibles (`tiempoMinutos`) y pulsa «Preparar guía» (`crear`, `GuiaView.tsx:297-315, 588`).
  3. **Crear el esqueleto de la guía:** el cliente llama a `api.crearGuia(actual.materia, idsOTemas, tiempoMinutos)` (`client.ts:426-433`) → `POST /api/guias` (`routers/guias.py:53-59`) → `GuiaOfflineService.crear` (`core/application/guias.py:59-117`):
     - Verifica que la asignatura tenga cronograma y que las clases seleccionadas no estén vacías (`guias.py:67-72`).
     - Para cada clase seleccionada, construye un objeto `ClaseGuia` con `clase_id`, `tema`, `fecha`, `objetivo`, `subtemas` (uniendo `contenido` y `subtemas_adicionales`), `estado_previo` y `contenido_md=""` (`guias.py:74-96`).
     - Calcula el nivel de `extension` (`"completa"`, `"estandar"` o `"sintesis"`) llamando a `elegir_extension(len(clases_guia), tiempo_minutos)` (`core/domain/guia_offline.py:28-53`).
     - Persiste la guía en `guias/guias.json` (`JsonGuiasRepo.guardar`, `infrastructure/persistence/json_guias.py:30-33`).
  4. **Generar cada clase paso a paso con IA:** a continuación, `empezar` (`GuiaView.tsx:276-295`) recorre secuencialmente los índices de la guía e invoca por cada uno `api.generarClaseGuia(g.id, i)` (`client.ts:435-440`) → `POST /api/guias/{guia_id}/clases/{indice}` (`routers/guias.py:91-101`) → `GuiaOfflineService.generar_clase(guia_id, indice)` (`core/application/guias.py:119-141`):
     - Recupera el contexto del plugin activo de la asignatura (`PluginsService.obtener_plugin`).
     - Construye el prompt adaptado a la `extension` (exigiendo incluir `<!-- RESPUESTAS -->` al final).
     - Genera el texto con `LLM_GeminiAdapter.generar_texto` y lo guarda en `objetivo.contenido_md` dentro de `guias/guias.json`.
  5. También se puede consultar una guía individual mediante `api.getGuia(id)` (`client.ts:413-416`) → `GET /api/guias/{guia_id}` (`routers/guias.py:62-64`) → `GuiaOfflineService.obtener` (`guias.py:144-148`).
- **Flujos alternativos y errores:**
  - Si no se seleccionan clases o la asignatura no tiene cronograma → `ValueError` → `HTTP 400` (`routers/guias.py:58`).
  - Si `guia_id` o `indice` no existen al generar una clase → `HTTP 404` (`routers/guias.py:95, 99`).
  - **Discrepancia asociada: DISC-10 (P27)** (resuelta en `2cead9e`).
- **Datos que lee y escribe:**
  - Lee: `schedules/<slug>_cronograma.json`, `plugins/<id>_context.md`.
  - Escribe: `guias/guias.json`. Consume cuota de Gemini por cada clase generada.
- **Resultado esperado:** Guía de estudio creada y persistida en `guias/guias.json` con el desarrollo teórico y las preguntas de autoevaluación de todas las clases seleccionadas.
- **Dónde está en el código y en la interfaz:**
  - Interfaz web: `frontend/src/views/GuiaView.tsx:207-612`, `frontend/src/lib/parciales.ts:19-67`.
  - Endpoints: `GET /api/guias`, `GET /api/guias/pendientes`, `POST /api/guias`, `GET /api/guias/{guia_id}`, `POST /api/guias/{guia_id}/clases/{indice}` (`apps/api/routers/guias.py:42-64, 91-101`).
  - Servicio y dominio: `GuiaOfflineService` (`core/application/guias.py:39-154`), `guia_offline.py` (`core/domain/guia_offline.py:13-131`).

CU-14: Previsualizar, imprimir y exportar una guía offline en PDF A5

- **Actor:** Estudiante.
- **Precondiciones:** Tener una guía offline en `guias/guias.json` con al menos una clase ya redactada (`contenido_md` no vacío).
- **Flujo principal:**
  1. **Vista limpia de lectura e impresión:** al abrir `/guia/:id/imprimir` (`frontend/src/views/GuiaImprimirView.tsx:21-157`), el navegador carga la guía con `api.getGuia(id)` (`líneas 26-31`), muestra el cuerpo de estudio de cada clase, agrupa todas las soluciones al final bajo «Respuestas a las preguntas» (`líneas 109-131`) y añade una hoja de registro en papel con casillas `Entendida` / `Con dudas` (`líneas 133-152`), permitiendo invocar `window.print()` (`línea 81`).
  2. **Exportación directa a PDF A5 desde el backend:** cuando el estudiante pulsa «Guardar PDF» en `GuiaView.tsx:76-82, 165-171` o `GuiaImprimirView.tsx:73-78`, el navegador solicita `GET /api/guias/{guia_id}/pdf` (`backend/src/modular_learning/apps/api/routers/guias.py:75-88`).
  3. El router invoca `GuiaOfflineService.exportar_pdf(guia_id)` (`core/application/guias.py:256-266`) → `ExportadorPdfPyMuPDF.exportar_guia(guia)` (`infrastructure/pdf/exportador_pdf.py:21-110`):
     - Convierte el Markdown de cada clase (`cuerpo` y `respuestas`) a HTML usando `MarkdownIt("commonmark")` (`exportador_pdf.py:26, 62-86`).
     - Construye un documento HTML completo con portada, índice de clases, desarrollo numerado, solucionario final y tabla de registro en papel (`_construir_html`, `exportador_pdf.py:50-110`).
     - Maqueta el HTML en páginas tamaño **A5** (`420 x 595 pt`, márgenes de `36 pt`) mediante `fitz.Story(html=html, user_css=_CSS)` y `fitz.DocumentWriter` (`exportador_pdf.py:16-17, 28-44`).
  4. El endpoint devuelve los bytes del PDF con `media_type="application/pdf"` y cabecera `Content-Disposition: attachment; filename="guia-<slug>.pdf"` (`routers/guias.py:82-87`), que el navegador descarga automáticamente.
- **Flujos alternativos y errores:**
  - Si `guia_id` no existe → `HTTP 404` (`routers/guias.py:38`).
  - Si la guía aún no tiene ninguna clase generada → `ValueError` en `GuiaOfflineService.exportar_pdf` (`guias.py:263-264`) → `HTTP 400` (`routers/guias.py:80`).
- **Datos que lee y escribe:**
  - Lee: `guias/guias.json`. No modifica ningún dato persistido.
- **Resultado esperado:** Descarga de un archivo `.pdf` en formato cuaderno A5 con las clases y el solucionario separado al final.
- **Dónde está en el código y en la interfaz:**
  - Interfaz web: `frontend/src/views/GuiaView.tsx:76-82, 165-171`, `frontend/src/views/GuiaImprimirView.tsx:21-157`.
  - Endpoint: `GET /api/guias/{guia_id}/pdf` (`apps/api/routers/guias.py:75-88`).
  - Servicio y adaptador: `GuiaOfflineService.exportar_pdf` (`core/application/guias.py:256-266`), `ExportadorPdfPyMuPDF` (`infrastructure/pdf/exportador_pdf.py:15-110`).

CU-15: Confirmar la revisión de una guía offline al volver

- **Actor:** Estudiante.
- **Precondiciones:** Tener una guía offline generada y haber estudiado sus clases fuera de línea.
- **Flujo principal:**
  1. En la vista de revisión `/guia/:id/revision` (`frontend/src/views/RevisionGuiaView.tsx:119-304`, accesible desde **Estudiar** › **Guía** o desde el recordatorio en **Hoy**, `HoyView.tsx:233-244`), el estudiante marca para cada clase uno de los tres estados (`"entendida"`, `"con_dudas"` o `"pendiente"`) y opcionalmente escribe sus dudas en el campo de texto (`RevisionGuiaView.tsx:235-289`). Mientras edita, los cambios se guardan automáticamente como borrador en `localStorage` bajo `ml:revision-guia:<id>` (`RevisionGuiaView.tsx:17, 136-148`).
  2. Al pulsar «Guardar revisión» (`guardar`, `RevisionGuiaView.tsx:150-167`), el cliente invoca `api.registrarRevisionGuia(guia.id, { clases: respuestas })` (`frontend/src/api/client.ts:442-449`) → `POST /api/guias/{guia_id}/revision` (`backend/src/modular_learning/apps/api/routers/guias.py:104-111`) → `GuiaOfflineService.registrar_revision(guia_id, revisiones)` (`core/application/guias.py:157-212`).
  3. Por cada clase incluida en `revisiones`:
     - Actualiza `clase.resultado` (`"entendida"`, `"con_dudas"`, `"pendiente"`) y `clase.dudas` (`guias.py:181-182`).
     - Si `resultado == "entendida"`:
       - Marca la clase como `"completado"` en el cronograma por `clase_id` (o por `tema` si no tiene `clase_id`) con `JsonCronogramaRepo.marcar_clase_completada` (`guias.py:184-187`).
       - Registra la clase en el repaso espaciado con `RepasoService.registrar_estudio_offline` (`guias.py:188-193`, `core/application/repaso.py:104-120`), creando tarjetas y un evento en `repaso/historial.json`.
     - Si `resultado != "entendida"` pero anteriormente estaba `"entendida"`, restaura `clase.estado_previo` en el cronograma (`guias.py:194-200`).
  4. Marca `guia.revisada = True`, asigna `guia.fecha_revision`, guarda la guía en `guias/guias.json` (`guias.py:202-204`) y

devuelve el resumen de clases completadas, con dudas y pendientes.

5. En el navegador se limpia el borrador de `localStorage` (`RevisionGuiaView.tsx:161`). Si alguna clase quedó marcada `"con_dudas"`, la pantalla de confirmación (`Resultado`, `RevisionGuiaView.tsx:21-117`) ofrece un enlace directo «Resolver con el tutor» que abre **Estudiar › Tutoría** con esa asignatura, `clase_id` y las dudas anotadas como enfoque (`líneas 68-75`).

- **Flujos alternativos y errores:**
  - Si `guia_id` no existe → `HTTP 404` (`routers/guias.py:38`).
  - Si se envía un `resultado` inválido → `ValueError` en `GuiaOfflineService.registrar_revision` (`guias.py:173-174`) → `HTTP 400` (`routers/guias.py:108`).
- **Datos que lee y escribe:**
  - Lee y escribe: `guias/guias.json`, `schedules/<slug>_cronograma.json`, `repaso/tarjetas.json`, `repaso/historial.json` y `localStorage`.
- **Resultado esperado:** Guía marcada como revisada, clases entendidas completadas en el cronograma y registradas en el repaso espaciado.
- **Dónde está en el código y en la interfaz:**
  - Interfaz web: `frontend/src/views/RevisionGuiaView.tsx:21-304`.
  - Endpoint: `POST /api/guias/{guia_id}/revision` (`apps/api/routers/guias.py:104-111`).
  - Servicio: `GuiaOfflineService.registrar_revision` (`core/application/guias.py:157-212`).

CU-16: Deshacer la confirmación de una guía offline o eliminar la guía

- **Actor:** Estudiante.
- **Precondiciones:** Tener una guía offline creada (y en el caso de deshacer, previamente confirmada).
- **Flujo principal:**
  1. **Deshacer confirmación:** en `RevisionGuiaView.tsx:169-184, 217-227`, el estudiante pulsa «Deshacer confirmación», lo que invoca `api.deshacerRevisionGuia(guia.id)` (`frontend/src/api/client.ts:451-456`) → `POST /api/guias/{guia_id}/deshacer` (`backend/src/modular_learning/apps/api/routers/guias.py:114-120`) → `GuiaOfflineService.deshacer_revision(guia_id)` (`core/application/guias.py:214-250`):
     - Por cada `ClaseGuia` cuyo `resultado` era `"entendida"`, restaura en el cronograma el estado que tenía antes de revisar la guía (`clase.estado_previo`, `"pendiente"` o `"completado"`) llamando a `JsonCronogramaRepo.marcar_clase` por `clase_id` o `tema` (`guias.py:222-238`).
     - Reinicia `clase.resultado = "pendiente"` y `clase.dudas = ""` en todas las clases de la guía (`guias.py:239-240`).
     - Pone `guia.revisada = False` y `guia.fecha_revision = None`, y guarda en `guias/guias.json` (`guias.py:241-243`).
  2. **Eliminar una guía:** en **Estudiar › Guía**, el estudiante pulsa «Borrar» en una guía de la lista (`eliminarGuia`, `GuiaView.tsx:179-189, 323-332`), llamando a `api.eliminarGuia(id)` (`client.ts:418-423`) → `DELETE /api/guias/{guia_id}` (`routers/guias.py:67-72`) → `GuiaOfflineService.eliminar_guia(guia_id)` (`guias.py:252-254`) → `JsonGuiasRepo.eliminar([guia_id])` (`infrastructure/persistence/json_guias.py:35-40`), que borra la guía de `guias/guias.json`.
- **Flujos alternativos y errores:**
  - Si `guia_id` no existe en cualquiera de las dos operaciones → `HTTP 404` (`routers/guias.py:70, 118`).
- **Datos que lee y escribe:**
  - Lee y escribe: `guias/guias.json` y `schedules/<slug>_cronograma.json`.
- **Resultado esperado:** Al deshacer, la guía vuelve al estado pendiente de confirmar y las clases del cronograma recuperan su `estado_previo`; al eliminar, la guía desaparece de `guias/guias.json`.
- **Dónde está en el código y en la interfaz:**
  - Interfaz web: `frontend/src/views/RevisionGuiaView.tsx:169-184, 217-227`, `frontend/src/views/GuiaView.tsx:179-189, 323-332`.
  - Endpoints: `POST /api/guias/{guia_id}/deshacer`, `DELETE /api/guias/{guia_id}` (`apps/api/routers/guias.py:67-72, 114-120`).
  - Servicio: `GuiaOfflineService.deshacer_revision` y `eliminar_guia` (`core/application/guias.py:214-254`).

CU-17: Configurar y actualizar plugins prácticos de contexto técnico (Web, CLI `ml up` y `snapshot.py`)

- **Actor:** Estudiante (en modo local o híbrido para escanear rutas del disco).
- **Precondiciones:** Contar con archivos de laboratorio o proyectos (`.pkt`/`.pka` para Cisco, `.json` para Node-RED o carpeta de código para Web).
- **Flujo principal (Interfaz Web):**
  1. **Listar plugins y su estado:** al abrir **Ajustes › Plugins** (`/ajustes?tab=plugins`, `frontend/src/views/AjustesView.tsx:14-63` → `frontend/src/views/PluginsView.tsx:45-345`, con redirección desde `/plugins` en `App.tsx:55`), `cargarPlugins`

(`PluginsView.tsx:59-71`) invoca `api.getPlugins()` (`client.ts:261-264`) → `GET /api/plugins` (`apps/api/routers/plugins.py:23-30`) → `PluginsService.listar_info_plugins()` (`core/application/plugins.py:95-123`), que devuelve para cada plugin registrado (`cisco`, `nodered`, `web`) su `id`, `nombre`, `materia`, `extensiones`, `carpeta` (leída de `config/plugins_config.json`) y `tiene_contexto` (si `plugins/<id>_context.md` tiene contenido).

2. **Configurar carpeta de un plugin:** el estudiante edita la ruta local en la tarjeta del plugin y pulsa «Guardar» (`handleGuardarCarpeta`, `PluginsView.tsx:124-146, 224-255`), llamando a `api.configurarPlugin({ plugin_id, carpeta })` (`client.ts:266-273`) → `POST /api/plugins/config` (`routers/plugins.py:33-39`) → `PluginsService.guardar_configuracion_plugin` (`plugins.py:86-93`) → `JsonPluginConfigRepo.guardar_configuracion` (`json_repositorios.py:412-426`), que persiste la ruta en `config/plugins_config.json`.
3. **Analizar carpeta o archivo y actualizar la memoria del plugin:** el estudiante pulsa «Sincronizar» en un plugin o «Sincronizar todos» (`handleActualizar`, `PluginsView.tsx:98-117, 167, 303`), invocando `api.actualizarPlugin(seleccion)` (`client.ts:275-282`) → `POST /api/plugins/actualizar` (`routers/plugins.py:42-60`) → `PluginsService.actualizar_plugin(seleccion, carpeta_override)` (`core/application/plugins.py:177-284`):
   - Procesa los archivos de laboratorio con `CiscoPlugin` (`infrastructure/plugins/cisco.py:213-388`), `NodeRedPlugin` (`infrastructure/plugins/nodered.py:31-245`) o `WebPlugin` (`infrastructure/plugins/web.py:304-429`), sintetiza el contexto técnico con Gemini y lo guarda en `plugins/<id>_context.md`.

- **Flujo alternativo 1 (CLI `ml up`):**
  1. El estudiante ejecuta `ml up` (`apps/cli/main.py:60-65` → `apps/cli/commands/up.py:15-56`), elige uno de los plugins registrados o todos, introduce opcionalmente una carpeta y se ejecuta `plugins_service.actualizar_plugin(seleccion, carpeta_override)`.
- **Flujo alternativo 2 (Script CLI `snapshot.py`):**
  1. El módulo `backend/src/modular_learning/apps/cli/commands/snapshot.py:10-128` permite recorrer una carpeta de código local (`.py`, `.js`, `.ts`, `.html`, `.css`, `.sql`, `.json`, `.md`), concatenar el árbol de directorios y el contenido de los archivos (truncando a 15.000 caracteres) y guardarlo directamente con `plugin.guardar_contexto_extraido(reporte)`. *(Véase **DISC-12**: este módulo no está conectado como comando en `apps/cli/main.py`).*
- **Errores:**
  - Selección de plugin no válida o carpeta sin archivos compatibles → mensaje de error o `HTTP 400` (`routers/plugins.py:55-58`).
- **Datos que lee y escribe:**
  - Lee: archivos `.pkt`, `.pka`, `.json` o código fuente en la ruta local configurada; `config/plugins_config.json`.
  - Escribe: `config/plugins_config.json` y `plugins/<id>_context.md`. Consume cuota de Gemini.
- **Resultado esperado:** Memoria técnica del plugin actualizada en `plugins/<id>_context.md` y lista para inyectarse automáticamente en las sesiones de tutoría y guías offline de la asignatura vinculada.
- **Dónde está en el código y en la interfaz:**
  - Interfaz web: `frontend/src/views/AjustesView.tsx:14-63`, `frontend/src/views/PluginsView.tsx:45-345`.
  - CLI: `ml up` (`apps/cli/commands/up.py:15-56`) y `apps/cli/commands/snapshot.py:10-128`.
  - Endpoints: `GET /api/plugins`, `POST /api/plugins/config`, `POST /api/plugins/actualizar` (`apps/api/routers/plugins.py:23-60`).
  - Servicio y adaptadores: `PluginsService` (`core/application/plugins.py:44-284`), `CiscoPlugin` (`infrastructure/plugins/cisco.py`), `NodeRedPlugin` (`infrastructure/plugins/nodered.py`), `WebPlugin` (`infrastructure/plugins/web.py`).

CU-18: Consultar y guardar ajustes de carpetas, clave de Gemini y apariencia

- **Actor:** Estudiante.
- **Precondiciones:** Servidor `ml-web` en ejecución.
- **Flujo principal:**
  1. **Consultar ajustes actuales:** al abrir **Ajustes › Carpetas y clave** (`/ajustes`, `frontend/src/views/AjustesView.tsx:14-63` → `frontend/src/views/ConfigView.tsx:23-221`, con redirección desde `/config` en `App.tsx:56`) o al iniciar vistas que consultan `en_nube`, el cliente llama a `api.getConfig()` (`frontend/src/api/client.ts:61-64`) → `GET /api/config` (`backend/src/modular_learning/apps/api/routers/config.py:17-23`) → `ConfiguracionService.obtener_ajustes()` (`core/application/configuracion.py:25-26`) → `JsonAjustesRepo.obtener_ajustes()` (`infrastructure/persistence/json_ajustes.py:39-46`). El router añade el booleano `en_nube = os.getenv("ML_EN_NUBE") == "1"` (`routers/config.py:21`).
  2. **Guardar rutas y clave de Gemini (solo en modo local/híbrido):** cuando `en_nube` es `false`, el estudiante edita `carpeta_material`, `ruta_obsidian` y `gemini_api_key` y pulsa «Guardar ajustes» (`handleSubmit`, `ConfigView.tsx:49-96, 206-214`). Se invoca `api.saveConfig(datos)` (`client.ts:66-73`) → `POST /api/config` (`routers/config.py:26-36`) → `ConfiguracionService.guardar_ajustes` (`configuracion.py:28-33`) → `JsonAjustesRepo.guardar_ajustes` (`json_ajustes.py:48-65`), que escribe `config/ajustes.json` y actualiza en caliente las propiedades de `settings` en memoria (`settings.py:26-33`). Si `en_nube` es `true`, `ConfigView.tsx:129-144` muestra el aviso «Entorno en la nube activo» y deshabilita los campos (`líneas 169, 188, 211`).
  3. **Cambiar apariencia (tema claro / oscuro):** desde el conmutador en el pie de la barra lateral (`frontend/src/components/layout/Sidebar.tsx:100-114`), el estudiante alterna entre modo claro y oscuro, persistiendo `ml-tema` en `localStorage` y

aplicando la clase `.dark` en `document.documentElement` (`frontend/src/lib/theme.ts:5-48`).

4. **Ubicación contextual de las acciones destructivas (`P20`, commit `01ff583`):** las acciones de vaciado ya no están en `ConfigView`, sino al pie del recurso que afectan bajo el desplegable «Zona peligrosa»: vaciar clases o eliminar asignatura en **Asignaturas › Clases** (`CU-01`, `CU-04`) y vaciar materiales y proyectos en **Asignaturas › Material** (`CU-09`).

- **Flujos alternativos y errores:**
  - Si `carpeta_material` o `ruta_obsidian` están vacíos al guardar → aviso de validación en el navegador (`ConfigView.tsx:52-64`).
- **Datos que lee y escribe:**
  - Lee y escribe: `config/ajustes.json` y `localStorage` (`ml-tema`).
- **Resultado esperado:** Rutas de trabajo, clave de API y tema visual actualizados inmediatamente sin necesidad de reiniciar el servidor.
- **Dónde está en el código y en la interfaz:**
  - Interfaz web: `frontend/src/views/AjustesView.tsx:14-63`, `frontend/src/views/ConfigView.tsx:23-221`, `frontend/src/components/layout/Sidebar.tsx:100-114`, `frontend/src/lib/theme.ts:5-48`.
  - Endpoints: `GET /api/config`, `POST /api/config` (`apps/api/routers/config.py:17-36`).
  - Servicio y repositorio: `ConfiguracionService` (`core/application/configuracion.py:10-33`), `JsonAjustesRepo` (`infrastructure/persistence/json_ajustes.py:15-65`).

CU-19: Consultar portada diaria «Hoy», progreso del semestre y comprobación de salud (`HoyView`, `ml stats`, `/api/salud`)

- **Actor:** Estudiante o servicio de monitoreo (Render healthcheck).
- **Precondiciones:** Ninguna (para `/api/salud` ni siquiera se requiere autenticación HTTP Basic).
- **Flujo principal:**
  1. **Portada diaria («Hoy», `frontend/src/views/HoyView.tsx:60-386`):** al abrir `/` (o `/progreso`, que redirige a `/` en `App.tsx:54`), `HoyView` reúne el estado del día y el progreso del semestre consultando (`HoyView.tsx:74-131`):
     - `GET /api/stats` (`api.getStats`) y `GET /api/materias` (`api.getMaterias`): obtiene el avance por asignatura (`clases_completadas`, `total_clases`, `porcentaje`), el `porcentaje_global` del semestre y `plugins_activos`.
     - `GET /api/materias/{materia}/clases` (`api.getProximasClases`): determina si hoy hay clases programadas en fecha (`clasesHoy`, `líneas 180-209`) o cuál es la próxima clase sugerida (`foco`, `líneas 155-162, 210-231`), ofreciendo el botón directo «Empezar tutoría» (`enlaceTutoria` con `clase.id`).
     - `GET /api/guias/pendientes` (`api.getGuiasPendientes`): si hay guías sin confirmar, muestra el recordatorio con enlace directo a `/guia/:id/revision` (`líneas 233-244`).
     - `GET /api/repaso/resumen` (`api.getResumenRepaso`): muestra el recordatorio de preguntas de repaso vencidas hoy (`líneas 333-341`).
     - `GET /api/config` → `GET /api/material/escanear` (`líneas 113-131`, actualizado en `P25` — commit `7da3dab`): solo cuando `!cfg.en_nube`, escanea las carpetas de materiales y, si detecta archivos pendientes, muestra el bloque «Tienes N archivos nuevos sin convertir en notas» con el botón «Convertir ahora» (`líneas 342-377`).
  2. **Progreso por asignatura y resumen del semestre (integrado en `HoyView` en `P20`, commit `01ff583`, y CLI `ml stats`):**
     - En la sección «Tus asignaturas» (`HoyView.tsx:246-331`), cada asignatura muestra su barra de progreso (`Regleta`, `líneas 40-58`), su contador `completadas / total clases` y el enlace a su siguiente clase pendiente. Debajo se incluye el desplegable `<details>` «Ver progreso completo» (`líneas 299-330`) con el avance global del semestre (`stats.porcentaje_global`), las preguntas dominadas (`resumen.dominadas`) y los plugins activos (`stats.plugins_activos`).
     - En la CLI, `ml stats` (`apps/cli/main.py:52-57` → `apps/cli/commands/stats.py:15-70`) invoca directamente `TelemetriaService.obtener_estadisticas()`.
     - `TelemetriaService.obtener_estadisticas()` (`core/application/telemetria.py:36-71`) recorre las asignaturas de `JsonMateriaRepo.cargar_materias()`, cuenta en cada cronograma el total de clases y las clases con `estado == "completado"`, calcula el porcentaje por asignatura y el `porcentaje_global`, y verifica qué plugins tienen memoria activa (`_contar_plugins_activos`, `telemetria.py:25-34`).
  3. **Comprobación de salud del servidor (`GET /api/salud`):**
     - Definido en `backend/src/modular_learning/apps/api/main.py:57-59`. Devuelve `{"ok": true}` con código `HTTP 200` y está exento del middleware de control de acceso (`apps/api/acceso.py:43-44`) para que plataformas PaaS como Render verifiquen que el contenedor está vivo.
- **Flujos alternativos y errores:**
  - Si no existen asignaturas ni cronogramas, `obtener_estadisticas` devuelve contadores en `0` (`telemetria.py:39-45`).
  - **Discrepancias asociadas: DISC-08 (`P25`)** (resuelta en `7da3dab`: `HoyView` no escanea carpetas cuando `en_nube` es `true`) y **DISC-12 (`D2` / `D3`)** (resuelta en documentación).
- **Datos que lee y escribe:**
  - Lee: `config/materias.json`, `schedules/*_cronograma.json`, `plugins/*_context.md`, `repaso/tarjetas.json`, `guias/guias.json`. No escribe datos (salvo si el estudiante pulsa «Convertir ahora» sobre materiales pendientes).

- **Resultado esperado:** Visión consolidada en una sola portada diaria del estudio de hoy, el avance curricular por asignatura y del semestre, y el estado de salud del servidor.
- **Dónde está en el código y en la interfaz:**
  - Interfaz web: `frontend/src/views/HoyView.tsx:40-386`.
  - CLI: `ml stats` (`backend/src/modular_learning/apps/cli/commands/stats.py:15-70`).
  - Endpoints: `GET /api/stats` (`apps/api/routers/stats.py:15-19`), `GET /api/salud` (`apps/api/main.py:57-59`).
  - Servicio: `TelemetriaService` (`core/application/telemetria.py:12-71`).

CU-20: Migrar datos entre versiones y sincronizar datos locales hacia Postgres en la nube (`scripts/migrar_datos.py` y `deploy/subir_datos.py`)

- **Actor:** Estudiante / Administrador del despliegue.
- **Precondiciones:** Acceso por terminal al entorno Python del proyecto; para `subir_datos.py`, tener definida la variable `DATABASE_URL` apuntando a PostgreSQL (Neon).
- **Flujo principal:**
  1. **Migración y saneamiento idempotente de datos (`scripts/migrar_datos.py:23-236`):**
     - Se ejecuta con `python scripts/migrar_datos.py` (o `--simular` para no escribir cambios). Opera sobre `almacen_por_defecto()` (`infrastructure/persistence/almacen.py:172-175`), por lo que funciona tanto contra `data/` local como contra Postgres si está definido `DATABASE_URL`.
     - **Paso 1 (Materias, `migrar_datos.py:38-70`):** lee `config/materias.json` (o `config/materias.txt` heredado), convierte entradas a objetos `Materia(slug, nombre)` y persiste `config/materias.json`.
     - **Paso 2 (Cronogramas, `migrar_datos.py:71-173`):** detecta colisiones en `schedules/*.json`, asigna `id` único a las clases que no lo tengan mediante `asignar_ids(clases)` (`core/domain/cronograma.py:17-34`) y renombra las claves de cronogramas antiguos al formato `schedules/<slug>_cronograma.json`.
     - **Paso 3 (Guías offline, `migrar_datos.py:174-225`):** recorre `guias/guias.json` y vincula `clase_id` en las clases de cada guía comparando `tema` + `fecha` (o `tema` único) contra el cronograma de la asignatura.
  2. **Subida de datos locales a Neon (`deploy/subir_datos.py:45-75`):**
     - Se ejecuta con `DATABASE_URL="postgresql://..." python deploy/subir_datos.py` (en modo simulación) o añadiendo `--subir` para escribir en la base de datos.
     - Recorre todas las claves de `AlmacenArchivos(settings.ruta_data())` excluyendo `examples/` y `README.md` (`subir_datos.py:31-42, 57-58`).
     - Al subir `config/ajustes.json`, reemplaza las rutas del PC por las rutas internas del contenedor Docker (`AJUSTES_NUBE` en `subir_datos.py:27-30`: `carpeta_material="/app/data/material"`, `ruta_obsidian="/app/data/boveda"`).
     - Escribe cada documento en `AlmacenPostgres(url)` mediante `destino.escribir(clave, contenido)` (`subir_datos.py:66-74`).
- **Flujos alternativos y errores:**
  - Si no se define `DATABASE_URL` al ejecutar `deploy/subir_datos.py`, termina con código de error `1` (`subir_datos.py:52-55`).
- **Datos que lee y escribe:**
  - `migrar_datos.py`: lee y escribe `config/materias.json`, `schedules/*_cronograma.json` y `guias/guias.json`.
  - `subir_datos.py`: lee todos los archivos de `data/` (salvo `examples/` y `README.md`) y escribe todas las filas en la tabla `documentos` de Postgres.
- **Resultado esperado:** Datos locales y remotos normalizados bajo el esquema `slug`/`id` actual y sincronizados en PostgreSQL para el despliegue en Render o uso híbrido.
- **Dónde está en el código y en la interfaz:**
  - Scripts CLI: `scripts/migrar_datos.py:1-243` y `deploy/subir_datos.py:1-79`.

---

5. Flujos de extremo a extremo del semestre

5.1. Flujo A: Empezar el semestre (Configuración inicial y planificación)

1. **Configurar carpetas y clave de API (CU-18):** el estudiante abre **Ajustes › Carpetas y clave** (`/ajustes`, `ConfigView`), define las rutas locales de su carpeta de materiales (`carpeta_material`) y su bóveda de Obsidian (`ruta_obsidian`), e introduce su `gemini_api_key`.
2. **Dar de alta las asignaturas (CU-01):** en **Asignaturas** (`/asignaturas`, `AsignaturasView`), crea las materias del semestre desde el botón «Nueva asignatura» del selector único verificando que sus nombres tengan la ortografía y tildes definitivas.
3. **Importar los sílabos oficiales (CU-02):** en **Asignaturas › Clases** (`SilabosView`) o ejecutando `ml es` en consola, extrae con IA las sesiones de cada sílabo `.pdf`, revisa la tabla de previsualización y guarda las clases de cada asignatura.
4. **Ajustar clases, cortes de parcial y revisar el calendario (CU-03):** en **Asignaturas › Clases**, edita o añade sesiones si el profesor hizo cambios al programa, define los cortes de parcial (`Parcial 1`, `Parcial 2`) eligiendo la última clase de cada corte en el

desplegable, y consulta la planificación mensual en la entrada **Calendario** (`/calendario`, `CalendarioView`).

5. **Configurar plugins prácticos (CU-17):** en **Ajustes › Plugins** (`/ajustes?tab=plugins`, `PluginsView`), configura las carpetas de laboratorio de cada plugin (`cisco`, `nodered`, `web`) para que el tutor disponga de contexto práctico durante el curso.
6. **Publicar datos en la nube si usa acceso móvil (CU-20):** si además despliega la aplicación en Render + Neon, ejecuta `python deploy/subir_datos.py --subir` para copiar el catálogo y los cronogramas iniciales a PostgreSQL.

5.2. Flujo B: Una semana normal de clases (Ingesta, estudio y repaso continuo)

1. **Revisar qué toca hoy (CU-19):** al abrir la aplicación en **Hoy** (`/`, `HoyView`), el estudiante identifica la clase del día o la próxima clase pendiente sugerida, las guías offline pendientes de confirmar, las preguntas de repaso vencidas y el avance por asignatura.
2. **Ingestar los archivos nuevos de la semana (CU-05 / CU-06):**
   - Si está en su PC (modo local o híbrido), coloca los documentos de la semana (`.pdf`, `.docx`, `.pptx`, `.txt`, `.md`) en `<carpeta_material>/<Asignatura>/` y pulsa «Convertir ahora» desde el aviso de **Hoy** o «Revisar carpetas» → «Convertir todo en notas» en **Asignaturas › Material** (`CU-05`, o ejecuta `ml im`), generando los apuntes en Obsidian y alineándolos con la clase correspondiente del cronograma.
   - Si está desde otro dispositivo o tiene un documento puntual, lo sube desde «Subir un archivo suelto» en **Asignaturas › Material** (`CU-06`).
3. **Estudiar la clase de la semana (dos modalidades complementarias bajo «Estudiar»):**
   - **Modalidad interactiva en pantalla (CU-10):** pulsa «Empezar tutoría» desde **Hoy** o desde el **Calendario** (que abre **Estudiar › Tutoría**, `TutoriaView`, o ejecuta `ml init`), recorre la explicación en *streaming* de cada subtema, plantea dudas en el chat lateral y al terminar pulsa «Terminar y guardar en Obsidian», lo que genera el resumen en su bóveda, marca la clase como completada por su `clase_id` y programa tarjetas de repaso.
   - **Modalidad desconectada / en papel (CU-13 → CU-14 → CU-15):** selecciona las clases de la semana en **Estudiar › Guía** (`/estudiar?tab=guia`, `GuiaView`), indica cuántos minutos tiene disponibles, genera la guía con IA (`CU-13`), guarda o imprime el cuaderno en **PDF A5** (`CU-14`) para estudiar sin pantallas y, al regresar, abre la revisión (`/guia/:id/revision`, `RevisionGuiaView`, `CU-15`) para marcar qué clases quedaron `Entendida` y cuáles `Con dudas` (abriendo estas últimas directamente en la tutoría).
4. **Consolidar la memoria a largo plazo (CU-11 y CU-12):** cierra la semana en **Estudiar › Repaso** (`/estudiar?tab=repaso`, `QuizView`) repasando las tarjetas vencidas del día (`CU-12`) o generando preguntas nuevas sobre los temas vistos (`CU-11`).

5.3. Flujo C: Preparación antes de un examen parcial

1. **Sincronizar entregables y laboratorios prácticos (CU-08 y CU-17):**
   - En **Asignaturas › Proyectos** (`/asignaturas?tab=proyectos`), convierte en notas o comprueba cambios de las fichas técnicas de los proyectos del parcial (`CU-08`).
   - En **Ajustes › Plugins** (`/ajustes?tab=plugins` o `ml up`), sincroniza las topologías de Packet Tracer (`cisco`), flujos de Node-RED (`nodered`) o código fuente (`web`) para que las explicaciones reflejen el estado real del proyecto (`CU-17`).
2. **Generar un cuaderno de repaso de todo el parcial (CU-13 y CU-14):** en **Estudiar › Guía** (`GuiaView`), pulsa el botón del corte de parcial en «Cortes de parcial» (que selecciona automáticamente todas las clases pendientes hasta la clase límite del corte), genera la guía y descarga el PDF A5 con solucionario al final.
3. **Practicar repaso espaciado y cuestionarios (CU-12, CU-11 y CU-19):**
   - En **Hoy** (`HoyView`), revisa el avance por asignatura y despliega «Ver progreso completo» para consultar el porcentaje global del semestre y las preguntas dominadas.
   - En **Estudiar › Repaso** (`QuizView`), ejecuta sesiones de repaso de tarjetas vencidas (`CU-12`) y cuestionarios nuevos (`CU-11`).
4. **Reforzar los fallos con el tutor (CU-10):** tras corregir cada cuestionario en `QuizView`, pulsa «Repasar este tema con el tutor» (`QuizView.tsx:267-275`), que inicia una sesión de tutoría enfocada exclusivamente en la pregunta fallada antes del examen.

---

6. Discrepancias detectadas y matriz de trazabilidad

6.1. Tabla de discrepancias (Comportamiento esperado vs. Comportamiento actual del código)

Cada discrepancia ha sido verificada directamente en el código fuente citando `archivo:línea` y contrastada con `docs/PENDIENTES.md`, `README.md`, `docs/ARCHITECTURE.md` y `docs/DESPLIEGUE.md`. Las discrepancias `DISC-01` a `DISC-06` y `DISC-08` a `DISC-12` han sido resueltas entre `P12` y `P29` / `D3`, quedando documentadas aquí con el commit que las cerró para trazabilidad histórica.

ID	Caso de uso	Comportamiento esperado	Comportamiento en el código (`archivo:línea`) y estado actual	Gravedad	Estado / Tarea en `PENDIENTES.md`

DISC-01	CU-01, CU-02	Al guardar un sílabo de una asignatura con tildes en `SilabosView`, debe quedar seleccionada la asignatura recién guardada.	**Resuelta en `bb1a30f` (P12):** `buscarMateriaPorClave` (`frontend/src/lib/asignaturas.ts:106-112`) normaliza con `clave()` (sin tildes) y `handleGuardarSilaboExtraido` (`SilabosView.tsx:512-514`) recarga el catálogo dejando seleccionada la asignatura recién guardada.	Media	**Resuelta (P12, commit `bb1a30f`)**
DISC-02	CU-04	Vaciar el cronograma debe dejar la asignatura en el catálogo sin clases, tal como promete el diálogo de confirmación en `SilabosView.tsx:265-267`.	**Resuelta en `bf2ac35` (P13):** `DELETE /api/materias/{materia}/cronograma` (`backend/src/modular_learning/apps/api/routers/materias.py:86`) invoca `servicio.eliminar_cronograma(materia, borrar_materia=False)`, eliminando solo `schedules/<slug>_cronograma.json` y conservando la asignatura en `config/materias.json`.	**Alta**	**Resuelta (P13, commit `bf2ac35`)**
DISC-03	CU-01, CU-02, CU-06, CU-10	Una asignatura escrita con otra grafía (mayúsculas, sin tildes) es la misma asignatura; el nombre visible del catálogo no cambia salvo al renombrar, y las notas en Obsidian usan siempre el nombre del catálogo.	**Resuelta en `a9b71aa` (P14):** 1) `JsonMateriaRepo.agregar_materia` (`infrastructure/persistence/json_repositorios.py:87-95`) verifica `if any(m.slug == materia.slug for m in materias): return` y no sobrescribe el nombre existente. 2) `SilaboService.guardar_cronograma` (`core/application/silabo.py:122-124`), `IngestaService._nombre_materia` (`core/application/ingesta.py:152-157, 167`) y `TutoriaService.iniciar_sesion` (`core/application/tutoria.py:135-138`) resuelven el nombre canónico mediante `resolver_materia` contra el catálogo.	**Alta**	**Resuelta (P14, commit `a9b71aa`)**
DISC-04	CU-02, CU-06	Al subir archivos en la web (`SilabosView` y `MaterialView`), la asignatura debe elegirse de una lista desplegable del catálogo sin pisar la selección previa del usuario.	**Resuelta en `1384372` (P15) y `2f8fe52` (P23):** `MaterialView.tsx:850-869` y `SilabosView.tsx:1068-1091` usan un `<select>` con las asignaturas del catálogo, y `resolverMateriaSubida` (`frontend/src/lib/asignaturas.ts:118-143`, `P23` — commit `2f8fe52`) conserva la asignatura ya elegida cuando el nombre del archivo no coincide con el catálogo.	Media	**Resuelta (P15 `1384372`, P23 `2f8fe52`)**
DISC-05	CU-05, CU-06	Las presentaciones `.pptx` deben poder ingestarse, incluidas las que contienen diapositivas con imágenes.	**Resuelta en `d40428d` (P16) y `faf23ac` (P24):** `IngestaService` incluye `.pptx` en `EXTENSIONES_MATERIAL` (`ingesta.py:29`), convierte a PDF temporal con `ConversorLibreOffice` usando perfil aislado `-env:UserInstallation` (`conversor_libreoffice.py:62-75`), aplica `python-pptx` como plan B con aviso visible en la web (`MaterialView.tsx:396-403`), habilita `.pptx` en el selector web (`MaterialView.tsx:891`) y rechaza presentaciones de solo imágenes (`< 200` caracteres útiles, `ingesta.py:39, 114-120`) cuando LibreOffice no está disponible.	**Alta**	**Resuelta (P16 `d40428d`, P24 `faf23ac`)**
DISC-06	CU-05, CU-06, CU-07	En la pestaña «Material», lo principal es revisar el historial de lo ingestado; subir un archivo suelto es ocasional y los resultados del escaneo solo deben mostrarse tras escanear.	**Resuelta en `2d6f757` (P17):** en `frontend/src/views/MaterialView.tsx:748-995`, la tarjeta «Materiales convertidos en notas» aparece como bloque principal (`líneas 748-834`), el formulario «Subir un archivo suelto» está plegado por defecto tras el botón «Subir un archivo» (`mostrarSubida`, `líneas 762-770, 837-938`), y los resultados del escaneo solo se renderizan cuando `escaneos.length > 0` (`líneas 954-995`).	Media	**Resuelta (P17, commit `2d6f757`)**
DISC-07	CU-05, CU-06	Extracción de tablas e imágenes en archivos `.docx`.	`LectorDocumentosLocal.extraer_texto` (`backend/src/modular_learning/infrastructure/documents/lector_documentos.py:51-55`) ejecuta `"\n".join(p.text for p in documento.paragraphs)` con `python-docx`, sin extraer tablas (`documento.tables`) ni imágenes embebidas, y `IngestaService._generar_contenido` (`core/application/ingesta.py:86-92`) envía los `.docx` como texto plano a `generar_texto`.	Media	**Abierta** (documentada para decisión futura)
DISC-08	CU-05, CU-08, CU-19	En modo nube (`ML_EN_NUBE=1`) no se deben escanear carpetas locales del servidor; en modo híbrido sí (`docs/DESPLIEGUE.md:34`).	**Resuelta en `7da3dab` (P25):** 1) `HoyView.tsx:113-131` consulta primero `api.getConfig()` y solo invoca `api.escanearMaterial()` cuando `!cfg.en_nube`. 2) `MaterialView.tsx:222, 313-336, 591` oculta la tarjeta «Tus carpetas de asignatura» y omite `escanearProyectos()` cuando `enNube` es `true`. 3) En el backend, `GET /api/material/escanear` (`apps/api/routers/material.py:57-58`) y `GET /api/proyectos/escanear` (`apps/api/routers/proyectos.py:37-38`) devuelven listas vacías sin tocar el disco cuando `os.getenv("ML_EN_NUBE") == "1"`.	Media	**Resuelta (P25, commit `7da3dab`)**

DISC-09	CU-10	Cerrar una tutoría debe marcar como completada la clase concreta (por su `id`), no otra clase que tenga el mismo nombre de tema.	**Resuelta en `511c3ab` (P26):** en la CLI (`ml init`), `pedir_datos_inicio` (`backend/src/modular_learning/apps/cli/ui/interaccion.py:83, 106`) conserva `clase_id_seleccionada` y la incluye en `"clase_id"`, y `apps/cli/commands/init.py:35` la envía a `tutoria_service.iniciar_sesion(..., clase_id=datos.get("clase_id"))`, igualando el comportamiento con la interfaz web (`TutoriaView.tsx:176`).	Media	**Resuelta (P26, commit `511c3ab`)**
DISC-10	CU-03, CU-13	Los cortes de parcial definidos en `SilabosView` deben permitir seleccionar todas las clases de ese parcial al crear una guía offline en `GuiaView`.	**Resuelta en `2cead9e` (P27):** en `SilabosView.tsx:745-777`, el formulario «Cortes de parcial» usa un `<select>` de las clases del cronograma y guarda `c.id ? String(c.id) : String(idx + 1)` en `hasta`. Tanto `SilabosView` como `GuiaView.tsx:345-363` usan `resolverIndiceHasta` y `describirCorte` (`frontend/src/lib/parciales.ts:19-67`), manteniendo retrocompatibilidad con cortes antiguos guardados por número o fecha.	Media	**Resuelta (P27, commit `2cead9e`)**
DISC-11	CU-01, CU-08, CU-09	Al eliminar un proyecto, eliminar una asignatura o vaciar materiales, todos los registros cruzados (`proyectos.json`, `registro_materiales.json`, cronogramas y notas en Obsidian) deben quedar consistentes.	**Resuelta en `58bf149` (P28):** 1) `ProyectosService.eliminar_proyecto` (`core/application/proyectos.py:55-64`) borra el proyecto de `config/proyectos.json`, elimina su entrada `[PROYECTO] ...` de `config/registro_materiales.json` y borra su ficha `.md` en Obsidian. 2) `MateriasService.eliminar_materia` (`core/application/materias.py:61-80`) elimina también todos los proyectos de la asignatura. 3) `IngestaService.vaciar_materiales` (`core/application/ingesta.py:314-349`) limpia `materiales_asociados` y `material_transversal` en todos los cronogramas (`JsonCronogramaRepo.limpiar_material_transversal`, `json_repositorios.py:298-305`).	Media	**Resuelta (P28, commit `58bf149`)**
DISC-12	CU-17, CU-19	La documentación técnica (`README.md`, `docs/ARCHITECTURE.md` y este manual) debe coincidir con las rutas reales (`data/config/`), las métricas reales de `ml stats` y las vistas actuales de `frontend/src/views/`.	**Resuelta en `e9fcd78` (D2) y actualizada en D3:** `README.md`, `docs/ARCHITECTURE.md` y este manual reflejan las rutas reales en `data/config/` y el menú de 5 entradas (`Hoy`, `Estudiar`, `Calendario`, `Asignaturas`, `Ajustes`) junto con las 11 vistas de `frontend/src/views/`.	Baja	**Resuelta (D2 `e9fcd78`, D3)**

6.2. Tabla de trazabilidad completa: Endpoints HTTP, Comandos CLI y Scripts → Casos de Uso

A. Endpoints HTTP del Backend (`backend/src/modular_learning/apps/api/`)

Están listados los **55 endpoints** expuestos por FastAPI (1 en `apps/api/main.py` y 54 distribuidos en los 11 routers de `apps/api/routers/`):

#	Método y Ruta HTTP	Router / Archivo (`archivo:línea`)	Servicio de Aplicación invocado (`archivo:línea`)	Caso de Uso
1	`GET /api/salud`	`apps/api/main.py:57-59`	Ninguno (healthcheck directo exento de auth)	**CU-19**
2	`GET /api/config`	`apps/api/routers/config.py:17-23`	`ConfiguracionService.obtener_ajustes` (`core/application/configuracion.py:25-26`)	**CU-18**
3	`POST /api/config`	`apps/api/routers/config.py:26-36`	`ConfiguracionService.guardar_ajustes` (`core/application/configuracion.py:28-33`)	**CU-18**
4	`GET /api/materias`	`apps/api/routers/materias.py:22-29`	`MateriasService.listar_materias` y `listar_catalogo` (`core/application/materias.py:35-41`)	**CU-01**
5	`POST /api/materias`	`apps/api/routers/materias.py:32-38`	`MateriasService.agregar_materia` (`core/application/materias.py:47-50`)	**CU-01**
6	`PUT /api/materias/{slug}`	`apps/api/routers/materias.py:41-51`	`MateriasService.renombrar_materia` (`core/application/materias.py:52-59`)	**CU-01**
7	`DELETE /api/materias/{slug}`	`apps/api/routers/materias.py:54-62`	`MateriasService.eliminar_materia` (`core/application/materias.py:61-80`)	**CU-01**
8	`GET /api/materias/cronogramas`	`apps/api/routers/materias.py:65-69`	`MateriasService.listar_cronogramas` (`core/application/materias.py:93-97`)	**CU-03, CU-19**

9	`GET /api/materias/{materia}/cronograma`	`apps/api/routers/materias.py:72-78`	`MateriasService.obtener_cronograma` (`core/application/materias.py:82-85`)	**CU-03**
10	`DELETE /api/materias/{materia}/cronograma`	`apps/api/routers/materias.py:81-89`	`MateriasService.eliminar_cronograma` (`core/application/materias.py:99-106`)	**CU-04**
11	`GET /api/materias/{materia}/clases`	`apps/api/routers/materias.py:92-97`	`MateriasService.obtener_proximas_clases` (`core/application/materias.py:87-91`)	**CU-03**, **CU-10**, **CU-19**
12	`POST /api/materias/{materia}/clases`	`apps/api/routers/materias.py:100-110`	`MateriasService.agregar_clase` (`core/application/materias.py:108-120`)	**CU-03**
13	`PUT /api/materias/{materia}/clases/{clase_id}`	`apps/api/routers/materias.py:113-126`	`MateriasService.editar_clase` (`core/application/materias.py:122-136`)	**CU-03**
14	`DELETE /api/materias/{materia}/clases/{clase_id}`	`apps/api/routers/materias.py:129-138`	`MateriasService.eliminar_clase` (`core/application/materias.py:138-149`)	**CU-03**
15	`PATCH /api/materias/{materia}/clases/{clase_id}/estado`	`apps/api/routers/materias.py:141-151`	`MateriasService.cambiar_estado_clase` (`core/application/materias.py:151-156`)	**CU-03**
16	`GET /api/materias/{materia}/parciales`	`apps/api/routers/materias.py:154-159`	`MateriasService.obtener_parciales` (`core/application/materias.py:158-162`)	**CU-03**, **CU-13**
17	`POST /api/materias/{materia}/parciales`	`apps/api/routers/materias.py:162-174`	`MateriasService.guardar_parciales` (`core/application/materias.py:164-173`)	**CU-03**
18	`POST /api/silabo/extraer`	`apps/api/routers/silabo.py:23-50`	`SilaboService.sugerir_nombre_materia` y `extraer_cronograma` (`core/application/silabo.py:73-118`)	**CU-02**
19	`POST /api/silabo/guardar`	`apps/api/routers/silabo.py:53-67`	`SilaboService.guardar_cronograma` (`core/application/silabo.py:120-129`)	**CU-02**
20	`GET /api/material/registrados`	`apps/api/routers/material.py:26-30`	`IngestaService.listar_materiales_ingestados` (`core/application/ingesta.py:230-232`)	**CU-07**
21	`POST /api/material/vaciar`	`apps/api/routers/material.py:33-39`	`IngestaService.vaciar_materiales` (`ingesta.py:314-349`) y `ProyectosService.vaciar_proyectos` (`proyectos.py:209-218`)	**CU-09**
22	`DELETE /api/material/{nombre_archivo:path}`	`apps/api/routers/material.py:42-51`	`IngestaService.eliminar_material` (`core/application/ingesta.py:234-262`)	**CU-07**
23	`GET /api/material/escanear`	`apps/api/routers/material.py:54-63`	`IngestaService.escanear` / `escanear_materia` (`core/application/ingesta.py:264-312, 351-352`)	**CU-05**, **CU-19**
24	`POST /api/material/sincronizar/stream`	`apps/api/routers/material.py:66-86`	`IngestaService.ingestar_pendientes_stream` (`core/application/ingesta.py:354-406`)	**CU-05**
25	`POST /api/material/ingestar`	`apps/api/routers/material.py:89-121`	`MateriasService.agregar_materia` y `IngestaService.ingestar_archivo` (`core/application/ingesta.py:159-228`)	**CU-06**
26	`GET /api/proyectos`	`apps/api/routers/proyectos.py:23-29`	`ProyectosService.listar_proyectos` (`core/application/proyectos.py:48-50`)	**CU-08**
27	`GET /api/proyectos/escanear`	`apps/api/routers/proyectos.py:32-50`	`ProyectosService.escanear_proyectos_en_carpeta` (`core/application/proyectos.py:66-74`)	**CU-08**
28	`GET /api/proyectos/{id_proyecto:path}/estado`	`apps/api/routers/proyectos.py:53-61`	`ProyectosService.verificar_estado_proyecto` (`core/application/proyectos.py:169-194`)	**CU-08**
29	`GET /api/proyectos/{id_proyecto:path}`	`apps/api/routers/proyectos.py:64-72`	`ProyectosService.obtener_proyecto` (`core/application/proyectos.py:52-53`)	**CU-08**
30	`POST /api/proyectos/ingestar`	`apps/api/routers/proyectos.py:75-90`	`ProyectosService.analizar_e_ingestar_proyecto` (`core/application/proyectos.py:126-167`)	**CU-08**
31	`POST /api/proyectos/{id_proyecto:path}/actualizar`	`apps/api/routers/proyectos.py:93-104`	`ProyectosService.actualizar_proyecto` (`core/application/proyectos.py:196-207`)	**CU-08**
32	`DELETE /api/proyectos/{id_proyecto:path}`	`apps/api/routers/proyectos.py:107-117`	`ProyectosService.eliminar_proyecto` (`core/application/proyectos.py:55-64`)	**CU-08**

33	`POST /api/tutoria/iniciar`	`apps/api/routers/tutoria.py:33-69`	`MateriasService.agregar_materia` y `TutoriaService.iniciar_sesion` (`core/application/tutoria.py:118-172`)	**CU-10**
34	`GET /api/tutoria/{sesion_id}/paso/{indice}/stream`	`apps/api/routers/tutoria.py:72-98`	`TutoriaService.explicar_paso_stream` (`core/application/tutoria.py:232-245`)	**CU-10**
35	`POST /api/tutoria/{sesion_id}/duda`	`apps/api/routers/tutoria.py:101-115`	`TutoriaService.responder_duda` (`core/application/tutoria.py:247-260`)	**CU-10**
36	`POST /api/tutoria/{sesion_id}/finalizar`	`apps/api/routers/tutoria.py:118-132`	`TutoriaService.finalizar_sesion` (`core/application/tutoria.py:262-296`)	**CU-10**
37	`GET /api/quiz/generar`	`apps/api/routers/quiz.py:31-42`	`EvaluacionService.generar_quiz` (`core/application/evaluacion.py:92-156`)	**CU-11**
38	`POST /api/quiz/evaluar`	`apps/api/routers/quiz.py:45-57`	`EvaluacionService.evaluar_respuestas` (`core/application/evaluacion.py:158-188`)	**CU-11**, **CU-12**
39	`GET /api/repaso/resumen`	`apps/api/routers/repaso.py:17-21`	`RepasoService.resumen` (`core/application/repaso.py:155-163`)	**CU-12**, **CU-19**
40	`GET /api/repaso/pendientes`	`apps/api/routers/repaso.py:24-33`	`RepasoService.preguntas_pendientes` (`core/application/repaso.py:128-153`)	**CU-12**
41	`GET /api/repaso/historial`	`apps/api/routers/repaso.py:36-42`	`RepasoService.historial` (`core/application/repaso.py:173-176`)	**CU-12**
42	`DELETE /api/repaso/tarjetas/{tarjeta_id}`	`apps/api/routers/repaso.py:45-53`	`RepasoService.eliminar_tarjeta` (`core/application/repaso.py:178-180`)	**CU-12**
43	`GET /api/guias`	`apps/api/routers/guias.py:42-44`	`GuiaOfflineService.listar` (`core/application/guias.py:150-151`)	**CU-13**
44	`GET /api/guias/pendientes`	`apps/api/routers/guias.py:47-50`	`GuiaOfflineService.pendientes_de_confirmar` (`core/application/guias.py:153-154`)	**CU-13**, **CU-15**, **CU-19**
45	`POST /api/guias`	`apps/api/routers/guias.py:53-59`	`GuiaOfflineService.crear` (`core/application/guias.py:59-117`)	**CU-13**
46	`GET /api/guias/{guia_id}`	`apps/api/routers/guias.py:62-64`	`GuiaOfflineService.obtener` (`core/application/guias.py:144-148`)	**CU-13**, **CU-14**
47	`DELETE /api/guias/{guia_id}`	`apps/api/routers/guias.py:67-72`	`GuiaOfflineService.eliminar_guia` (`core/application/guias.py:252-254`)	**CU-16**
48	`GET /api/guias/{guia_id}/pdf`	`apps/api/routers/guias.py:75-88`	`GuiaOfflineService.exportar_pdf` (`core/application/guias.py:256-266`)	**CU-14**
49	`POST /api/guias/{guia_id}/clases/{indice}`	`apps/api/routers/guias.py:91-101`	`GuiaOfflineService.generar_clase` (`core/application/guias.py:119-141`)	**CU-13**
50	`POST /api/guias/{guia_id}/revision`	`apps/api/routers/guias.py:104-111`	`GuiaOfflineService.registrar_revision` (`core/application/guias.py:157-212`)	**CU-15**
51	`POST /api/guias/{guia_id}/deshacer`	`apps/api/routers/guias.py:114-120`	`GuiaOfflineService.deshacer_revision` (`core/application/guias.py:214-250`)	**CU-16**
52	`GET /api/plugins`	`apps/api/routers/plugins.py:23-30`	`PluginsService.listar_info_plugins` (`core/application/plugins.py:95-123`)	**CU-17**
53	`POST /api/plugins/config`	`apps/api/routers/plugins.py:33-39`	`PluginsService.guardar_configuracion_plugin` (`core/application/plugins.py:86-93`)	**CU-17**
54	`POST /api/plugins/actualizar`	`apps/api/routers/plugins.py:42-60`	`PluginsService.actualizar_plugin` (`core/application/plugins.py:177-284`)	**CU-17**
55	`GET /api/stats`	`apps/api/routers/stats.py:15-19`	`TelemetriaService.obtener_estadisticas` (`core/application/telemetria.py:36-71`)	**CU-19**

B. Comandos de la CLI y Scripts Operativos (`apps/cli/`, `scripts/`, `deploy/`)

#	Comando / Script	Punto de entrada (`archivo:línea`)	Implementación (`archivo:línea`)	Servicio(s) invocado(s)	Caso de Uso

1	`ml init`	`apps/cli/main.py:20-25`	`apps/cli/commands/init.py:13-95` y `apps/cli/ui/interaccion.py:33-126`	`MateriasService`, `TutoriaService` (`iniciar_sesion` con `clase_id`, `explicar_paso_stream`, `responder_duda`, `finalizar_sesion`)	**CU-10**
2	`ml es`	`apps/cli/main.py:28-33`	`apps/cli/commands/es.py:12-83`	`SilaboService` (`sugerir_nombre_materia`, `extraer_cronograma`, `guardar_cronograma`)	**CU-02**
3	`ml im`	`apps/cli/main.py:36-41`	`apps/cli/commands/im.py:12-106`	`MateriasService.listar_materias`, `IngestaService` (`escanear_materia`, `ingestar_archivo`), `ProyectosService.analizar_e_ingestar_proyecto`	**CU-05**, **CU-08**
4	`ml quiz`	`apps/cli/main.py:44-49`	`apps/cli/commands/quiz.py:14-81`	`EvaluacionService` (`generar_quiz`, `evaluar_respuestas`), `RepasoService.registrar_resultados`	**CU-11**
5	`ml stats`	`apps/cli/main.py:52-57`	`apps/cli/commands/stats.py:15-70`	`TelemetriaService.obtener_estadisticas`	**CU-19**
6	`ml up`	`apps/cli/main.py:60-65`	`apps/cli/commands/up.py:15-56`	`PluginsService` (`plugins`, `actualizar_plugin`)	**CU-17**
7	`python -m modular_learning.apps.cli.commands.snapshot`	`apps/cli/commands/snapshot.py:124-125` (no registrado en `apps/cli/main.py`)	`apps/cli/commands/snapshot.py:10-128`	`MateriasService.listar_materias`, `PluginsService` (`obtener_plugin`, `guardar_contexto_extraido`)	**CU-17**
8	`ml-web`	`backend/pyproject.toml:34`	`apps/api/main.py:135-145`	Servidor FastAPI / Uvicorn (expone todos los endpoints HTTP #1 a #55)	**CU-01** a **CU-19**
9	`python scripts/migrar_datos.py [--simular]`	`scripts/migrar_datos.py:239-240`	`scripts/migrar_datos.py:23-236`	`almacen_por_defecto` (`AlmacenDocumentos`), `Materia`, `asignar_ids`	**CU-20**
10	`python deploy/subir_datos.py [--subir]`	`deploy/subir_datos.py:75-76`	`deploy/subir_datos.py:45-73`	`AlmacenArchivos`, `AlmacenPostgres`	**CU-20**